

第一篇 Hudi 快速上手

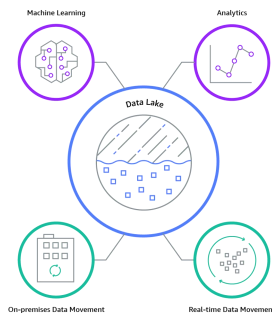
1.1. 数据湖简介

1.1.1. 什么是数据湖

数据湖或 hub 的概念最初是由大数据厂商提出的,表面上看,数据都是承载在基于可向扩展的 HDFS 廉价存储硬件之上的。但数据量越大,越需要各种不同种类的存储。最终,所有的企业数据都可以被认为是大数据,但并不是所有的企业数据都是适合存放在廉价的 HDFS 集群之上的。

维基上对它的解释:数据湖(Data Lake)是一个以原始格式存储数据的存储库或系统。它按原样存储数据,而无需事先对数据进行结构化处理。一个数据湖可以存储结构化数据(如关系型数据库中的表),半结构化数据(如 CSV、日志、XML、JSON),非结构化数据(如电子邮件、文档、PDF)和二进制数据(如图形、音频、视频)。

数据湖是一个集中式存储库,允许您以任意规模存储所有结构化和非结构化数据。您可以按原样存储数据(无需先对数据进行结构化处理),并运行不同类型的分析 - 从控制面板和可视化到大数据处理、实时分析和机器学习,以指导做出更好的决策。



数据应该具备如下功能

✓ 数据集成能力:

需要具备把各种数据源接入集成到数据湖中的能力。数据湖的存储也应该是多样的,比如 HDFS、HIVE、HBASE 等等。

✓ 数据治理能力:

治理能力的核心是维护好数据的元数据(metadata)。强制要求所有进入数据湖的数据必须提供相关元数据,应该作为最低限度的治理管控。没有元数据,数据湖就面临成为数据沼泽的风险。更丰富的功能还包括:

- ✧ 自动提取元数据,并根据元数据对数据进行分类,形成数据目录。
- ✧ 自动对数据目录进行分析,可以基于 AI 和机器学习的方法,发现数据之间的关系。
- ✧ 自动建立数据之间血缘关系图。
- ✧ 跟踪数据的使用情况,以便将数据作为产品,形成数据资产。

✓ 数据搜索和发现能力:

如果把整个互联网想象成一个巨大的数据湖。那么,之所以人们可以这么有效的利用这个湖中的数据,就是因为有了 Google 这样的搜索引擎。人们可以通过搜索,方便地找到他们想要的数 据,进而进行分析。搜索能力是数据湖的十分重要的能力。

✓ **数据安全管控能力:**

对数据的使用权限进行管控,对敏感数据进行脱敏或加密处理,也是数据湖能商用所必须具备的能力。

✓ **数据质量检验能力:**

数据质量是分析正确的关键。因此必须对进入数据湖中的数据的质量情况进行检验。及时发现数据湖中数据质量的问题。为有效的数据探索提供保障。

✓ **自助数据探索能力:**

应该具备一系列好用的数据分析工具,以便各类用户可以对数据湖中的数据进行自助探索。包括:

- ✧ 支持对流、NoSQL、图等多种存储库的联合分析能力
- ✧ 支持交互式的大数据 SQL 分析
- ✧ 支持 AI、机器学习分析
- ✧ 支持类似 OLAP 的 BI 分析
- ✧ 支持报表的生成

1.1.2. 为什么要数据湖

通过数据成功创造商业价值的组织将胜过同行。Aberdeen 的一项调查表明,实施数据湖的组织比同类公司在有机收入增长方面高出 9%。这些领导者能够进行新类型的分析,例如通过日志文件、来自点击流的数据、社交媒体以及存储在数据湖中的互联网连接设备等新来源的机器学习。这有助于他们通过吸引和留住客户、提高生产力、主动维护设备以及做出明智的决策来更快地识别和应对业务增长机会。

数据湖和数据仓库对比

数据仓库是一个优化的数据库,用于分析来自事务系统和业务线应用程序的关系数据。事先定义数据结构和 Schema 以优化快速 SQL 查询,其中结果通常用于操作报告和分析。数据经过了清理、丰富和转换,因此可以充当用户可信任的“单一信息源”。

数据湖有所不同,因为它存储来自业务线应用程序的关系数据,以及来自移动应用程序、IoT 设备和社交媒体的非关系数据。捕获数据时,未定义数据结构或 Schema。这意味着您可以存储所有数据,而不需要精心设计也无需知道将来您可能需要哪些问题的答案。您可以对数据使用不同类型的分析(如 SQL 查询、大数据分析、全文搜索、实时分析和机器学习)来获得见解

特征	数据仓库	数据湖
数据	来自事务系统、运营数据库和业务线应用程序的关系数据	来自 IoT 设备、网站、移动应用程序、社交媒体和企业应用程序的非关系和关系数据
Schema	设计在数据仓库实施之前(写入型 Schema)	写入在分析时(读取型 Schema)
性价比	更快查询结果会带来较高存储成本	更快查询结果只需较低存储成本
数据质量	可作为重要事实依据的高度监管数据	任何可以或无法进行监管的数据(例如原始数据)

用户	业务分析师	数据科学家、数据开发人员和业务分析师（使用监管数据）
分析	批处理报告、BI 和可视化	机器学习、预测分析、数据发现和分析

数据湖和分析解决方案的基本要素

组织构建数据湖和分析平台时，他们需要考虑许多关键功能，包括：

数据移动

数据湖允许您导入任何数量的实时获得的数据。您可以从多个来源收集数据，并以其原始形式将其移入到数据湖中。此过程允许您扩展到任何规模的数据，同时节省定义数据结构、Schema 和转换的时间。

安全地存储和编目数据

数据湖允许您存储关系数据（例如，来自业务线应用程序的运营数据库和数据）和非关系数据（例如，来自移动应用程序、IoT 设备和社交媒体的运营数据库和数据）。它们还使您能够通过过对数据进行爬网、编目和建立索引来了解湖中的数据。最后，必须保护数据以确保您的数据资产受到保护。

分析

数据湖允许组织中的各种角色（如数据科学家、数据开发人员和业务分析师）通过各自选择的分析工具和框架来访问数据。这包括 Apache Hadoop、Presto 和 Apache Spark 等开源框架，以及数据仓库和商业智能供应商提供的商业产品。数据湖允许您运行分析，而无需将数据移至单独的分析系统。

机器学习

数据湖将允许组织生成不同类型的见解，包括报告历史数据以及进行机器学习（构建模型以预测可能的结果），并建议一系列规定的行动以实现最佳结果。

数据湖的价值

能够在更短的时间内从更多来源利用更多数据，并使用户能够以不同方式协同处理和分析数据，从而做出更好、更快的决策。数据湖具有增值价值的示例包括：

改善客户互动

数据湖可以将来自 CRM 平台的客户数据与社交媒体分析相结合，有一个包括购买历史记录和事故单的营销平台，使企业能够了解最有利可图的客户群、客户流失的原因以及将提升忠诚度的促销活动或奖励。

改善研发创新选择

数据湖可以帮助您的研发团队测试其假设，改进假设并评估结果 - 例如在产品设计中选择正确的材料从而提高性能，进行基因组研究从而获得更有效的药物，或者了解客户为不同属性付费的意愿。

提高运营效率

物联网 (IoT) 引入了更多方式来收集有关制造等流程的数据，包括来自互联网连接设备的实时数据。使用数据湖，可以轻松地将数据存储在 IoT 数据进行分析，以发现降低运营成本和提高质量的方法。

1.2. Hudi 简介

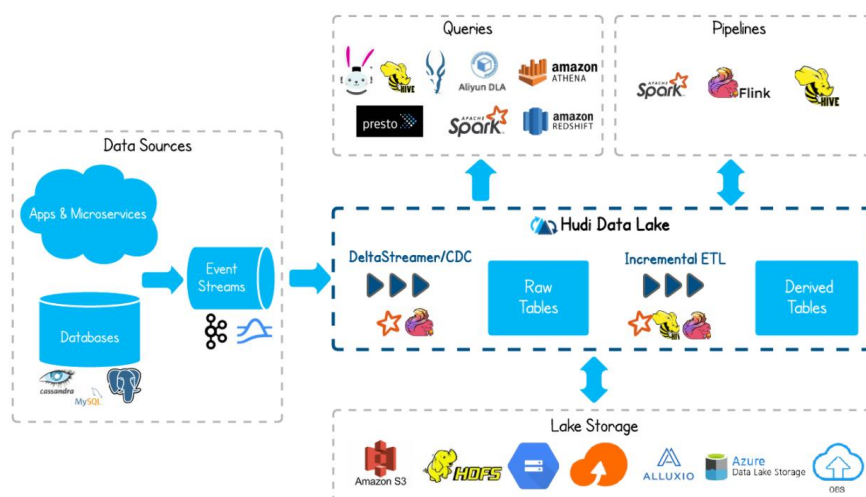


Apache Hudi!

Hudi brings transactions, record-level updates/deletes and change streams to data lakes!

[Latest Releases](#)[Documentation](#)

Hudi 是一个丰富的平台，用于在自我管理的数据库层上构建具有增量数据管道的流数据湖，同时针对湖引擎和常规批处理，并且进行了优化。



具备如下功能

- ✧ Upserts, Deletes with fast, pluggable indexing.
- ✧ Incremental queries, Record level change streams
- ✧ Transactions, Rollbacks, Concurrency Control.
- ✧ SQL Read/Writes from Spark, Presto, Trino, Hive & more
- ✧ Automatic file sizing, data clustering, compactions, cleaning.
- ✧ Streaming ingestion, Built-in CDC sources & tools.
- ✧ Built-in metadata tracking for scalable storage access.
- ✧ Backwards compatible schema evolution and enforcement.

- (1) 快速 upsert,可插入索引
- (2) 以原子方式操作数据并具有回滚功能
- (3) 写入器之和查询之间的快照隔离
- (4) savepoint 用户数据恢复的保存点
- (5) 管理文件大小,使用统计数据布局
- (6) 异步压缩行列数据
- (7) 具有时间线来追踪元数据血统
- (8) 通过聚类优化数据集

1.3. Hudi 构建

1.3.1. linux 系统安装 maven

1) 下载 maven

<https://gitee.com/niubi123/doit-hudi/blob/master/dev/apache-maven-3.8.3-bin.tar.gz>

2) 上传到 linux 机器

```
-rw-r--r--. 1 root root 9042049 Oct 18 14:30 apache-maven-3.8.3-bin.tar.gz
drwxrwxr-x. 10 root root 156 Oct 25 20:28 flink-1.13.1
```

3) 解压

`tar -zxvf apache-maven-3.8.3-bin.tar.gz`

4) 配置下载镜像

```
drwxr-xr-x. 2 root root    37 Sep 28 02:25 logging
-rw-r--r--. 1 root root 10742 Sep 28 02:25 settings.xml
-rw-r--r--. 1 root root  3747 Sep 28 02:25 toolchains.xml
[root@doit02 conf]# pwd
/opt/apps/apache-maven-3.8.3/conf
```

<!-- 添加阿里云镜像-->

```
<mirror>
  <id>nexus-aliyun</id>
  <mirrorOf>central</mirrorOf>
  <name>Nexus aliyun</name>
  <url>http://maven.aliyun.com/nexus/content/groups/public</url>
</mirror>
```

5) 配置系统环境变量

vi /etc/profile

```
export MAVEN_HOME=/opt/apps/maven-3.8.3
```

6) 测试

```
[root@doit01 apps]# mvn -version
```

```
Apache Maven 3.8.3 (ff8e977a158738155dc465c6a97ffaf31982d739)
```

```
Maven home: /opt/apps/maven-3.8.3
```

```
Java version: 1.8.0_191, vendor: Oracle Corporation, runtime: /opt/apps/jdk1.8.0_191/jre
```

1.3.2. linux 系统安装 git

1) yum list | grep git

2) yum -y install git.x86_64

3) git --version

1.3.3. Hudi 项目克隆

<https://gitee.com/niubi123/doit-hudi/blob/master/dev/hudi-master.tar.gz>

下载地址, 将下载的压缩包上传值虚拟

1.3.4. Hudi 项目构建

可以修改自己使用的组件版本号, 比如 flink, spark, hive 等等

```
<hive.groupid>org.apache.hive</hive.groupid>
<hive.version>3.1.2</hive.version>
<hive.exec.classifier>core</hive.exec.classifier>
<metrics.version>4.1.1</metrics.version>
<orc.version>1.6.0</orc.version>
<airlift.version>0.16</airlift.version>
<prometheus.version>0.8.0</prometheus.version>
<http.version>4.4.1</http.version>
<spark.version>${spark2.version}</spark.version>
<sparkbundle.version>${spark2bundle.version}</sparkbundle.version>
<flink.version>1.13.1</flink.version>
<spark2.version>2.4.4</spark2.version>
<spark3.version>3.0.0</spark3.version>
```


进入到项目的根目录下执行

```
mvn clean package -DskipTests -Dspark3 -Dscala-2.12 -Ddrat.skip=true -Pflink-bundle-shade-hive3
```

```
INFO] -----
INFO] BUILD SUCCESS
INFO] -----
INFO] Total time: 07:12 min
INFO] Finished at: 2021-10-29T22:03:52+08:00
INFO] -----
drwxr-xr-x. 4 197609 197121 46 Oct 29 22:03 hudi-flink-bundle
drwxr-xr-x. 4 197609 197121 46 Oct 29 22:00 hudi-hadoop-mr-bundle
drwxr-xr-x. 4 197609 197121 46 Oct 29 22:00 hudi-hive-sync-bundle
drwxr-xr-x. 4 197609 197121 46 Oct 29 22:01 hudi-integ-test-bundle
drwxr-xr-x. 4 197609 197121 46 Oct 29 22:03 hudi-kafka-connect-bundle
drwxr-xr-x. 4 197609 197121 46 Oct 29 22:00 hudi-presto-bundle
drwxr-xr-x. 4 197609 197121 46 Oct 29 22:00 hudi-spark-bundle
drwxr-xr-x. 4 197609 197121 101 Oct 29 22:01 hudi-timeline-server-bundle
drwxr-xr-x. 4 197609 197121 46 Oct 29 21:59 hudi-utilities-bundle
-rw-r--r--. 1 197609 197121 2206 Oct 28 16:16 README.md
[root@doit01 packaging]#
```

1.4. spark 读写 Hudi

将 hudi 整合 spark 的 jar 包复制到 spark 的 jars 包下

```
cp /data/hudi-0.9.0/packaging/hudi-spark-bundle/target/hudi-spark3-bundle_2.12-0.9.0.jar
./ /opt/apps/spark-3.0.0/jars/
```

1.4.1. spark-shell 入门

1.4.1.1. 启动 sparkshell

```
spark-shell --jars
/data/hudi-0.9.0/packaging/hudi-spark-bundle/target/hudi-spark3-bundle_2.12-0.9.0.jar --packages org.apache.spark:spark-avro_2.12:3.0.0 --conf
'spark.serializer=org.apache.spark.serializer.KryoSerializer'
```

1.4.1.2. 导入依赖

```
import org.apache.hudi.QuickstartUtils._
import scala.collection.JavaConversions._
import org.apache.spark.sql.SaveMode._
import org.apache.hudi.DataSourceReadOptions._
import org.apache.hudi.DataSourceWriteOptions._
import org.apache.hudi.config.HoodieWriteConfig._
import org.apache.spark.sql.{DataFrame, SparkSession}
import org.apache.spark.sql.types.{DataTypes, StructType}
```

1.4.1.3. 加载静态数据 , 生成 DF

```
val schema = new StructType().add("id", DataTypes.StringType).add("name",
DataTypes.StringType).add("age", DataTypes.).add("gender",
DataTypes.StringType).add("city", DataTypes.StringType)
//创建提交时间
val commitTime = System.currentTimeMillis().toString
//生成提交时间
val df: DataFrame =
spark.read.schema(schema).csv("/data/user.txt").withColumn("ts" ,
lit(commitTime))
```

1.4.1.4. 将 DF 数据写入到 hudi

```
df.write.format("hudi").
  options(getQuickstartWriteConfigs).
  option(PRECOMBINE_FIELD.key(), "ts").
  option(RECORDKEY_FIELD.key(), "id").
  option(PARTITIONPATH_FIELD.key(), "city").
  option(TBL_NAME.key(), "hudi_test01").
  mode(Overwrite).
  save("hdfs://doit01:8020/hudi_dw/hudi_test01")
```

1.4.1.5. 读取数据 展示数据

```
val resDF = spark.read.format("hudi").load("hdfs://doit01:8020/hudi_dw/hudi_test01")
//展示数据
resDF.show
// 使用 sparkSQL 的方式查询数据
resDF.createOrReplaceTempView("tb_test01")
spark.sql("select * from tb_test01").show
```

1.4.2. sparksql-shell 入门

1.4.2.1. 启动 sparksql 客户端

```
spark-sql --jars
/data/hudi-0.9.0/packaging/hudi-spark-bundle/target/hudi-spark3-bundle
_2.12-0.9.0.jar --packages
org.apache.hudi:hudi-spark3-bundle_2.12:0.9.0,org.apache.spark:spark-a
vro_2.12:3.0.1 \
--conf 'spark.serializer=org.apache.spark.serializer.KryoSerializer' \
--conf
```

```
'spark.sql.extensions=org.apache.spark.sql.hudi.HoodieSparkSessionExtension'
```

1.4.2.2. 在 sparksql 中创建表导入查询

```
-- 创建表
drop table if exists tb_test02 ;
create table if not exists tb_test02 (
  id int,
  name string,
  age int ,
  gender string ,
  city string
) using hudi
location 'hdfs://doit01:8020/hudi_dw/hudi_test02'
options (
  type = 'cow',
  primaryKey = 'id'
);
-- 向表中插入数据
insert into tb_test02 select 1,'zss',23,'M','SH' ;
-- 查询数据
select * from tb_test02 ;
-- 删除数据
delete from tb_test02 where id = 1;
```

1.4.3. idea 开发工具编程入门

1.4.3.1. 添加依赖

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>org.example</groupId>
  <artifactId>hudi_ch</artifactId>
  <version>1.0-SNAPSHOT</version>
  <properties>
    <maven.compiler.source>8</maven.compiler.source>
```



```
<maven.compiler.target>8</maven.compiler.target>
</properties>
<dependencies>
  <dependency>
    <groupId>com.alibaba</groupId>
    <artifactId>fastjson</artifactId>
    <version>1.2.75</version>
  </dependency>
  <dependency>
    <groupId>org.scala-lang</groupId>
    <artifactId>scala-library</artifactId>
    <version>2.12.10</version>
  </dependency>
  <dependency>
    <groupId>org.apache.hudi</groupId>
    <artifactId>hudi-spark-bundle_2.12</artifactId>
    <version>0.9.0</version>
  </dependency>
  <!-- https://mvnrepository.com/artifact/joda-time/joda-time -->
  <dependency>
    <groupId>joda-time</groupId>
    <artifactId>joda-time</artifactId>
    <version>2.9.9</version>
  </dependency>

  <!-- https://mvnrepository.com/artifact/org.apache.spark/spark-core -->
  <!-- <dependency>
    <groupId>org.apache.spark</groupId>
    <artifactId>spark-core_2.12</artifactId>
    <version>3.0.0</version>
  </dependency>-->

  <!-- https://mvnrepository.com/artifact/org.apache.spark/spark-streaming -->
  <dependency>
    <groupId>org.apache.spark</groupId>
    <artifactId>spark-streaming_2.12</artifactId>
    <version>3.0.0</version>
  </dependency>
  <dependency>
    <groupId>org.apache.spark</groupId>
    <artifactId>spark-sql_2.12</artifactId>
    <version>3.0.0</version>
  </dependency>
```

```
<dependency>
  <groupId>com.doitedu</groupId>
  <artifactId>hudi_spark</artifactId>
  <version>12</version>
  <scope>system</scope>
  <systemPath>${project.basedir}/lib/hudi-spark3-bundle_2.12-0.9.0.jar</systemPath>
</dependency>
</dependencies>
<!-- 依赖下载国内镜像库 -->
<repositories>
  <repository>
    <id>nexus-aliyun</id>
    <name>Nexus aliyun</name>
    <layout>default</layout>
    <url>http://maven.aliyun.com/nexus/content/groups/public</url>
    <snapshots>
      <enabled>false</enabled>
      <updatePolicy>never</updatePolicy>
    </snapshots>
    <releases>
      <enabled>true</enabled>
      <updatePolicy>never</updatePolicy>
    </releases>
  </repository>
</repositories>

<!-- maven 插件下载国内镜像库 -->
<pluginRepositories>
  <pluginRepository>
    <id>ali-plugin</id>
    <url>http://maven.aliyun.com/nexus/content/groups/public/</url>
    <snapshots>
      <enabled>false</enabled>
      <updatePolicy>never</updatePolicy>
    </snapshots>
    <releases>
      <enabled>true</enabled>
      <updatePolicy>never</updatePolicy>
    </releases>
  </pluginRepository>
</pluginRepositories>
<build>
  <plugins>
    <!-- 指定编译 java 的插件 -->
```

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <version>3.5.1</version>
  <configuration>
    <source>1.8</source>
    <target>1.8</target>
  </configuration>
</plugin>

<!-- 指定编译 scala 的插件 -->
<plugin>
  <groupId>net.alchim31.maven</groupId>
  <artifactId>scala-maven-plugin</artifactId>
  <version>3.2.2</version>
  <executions>
    <execution>
      <goals>
        <goal>compile</goal>
        <goal>testCompile</goal>
      </goals>
      <configuration>
        <args>
          <arg>-dependencyfile</arg>
          <arg>${project.build.directory}/.scala_dependencies</arg>
        </args>
      </configuration>
    </execution>
  </executions>
</plugin>
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <configuration>
    <source>1.8</source>
    <target>1.8</target>
  </configuration>
</plugin>
<plugin>
  <artifactId>maven-assembly-plugin</artifactId>
  <configuration>
    <descriptorRefs>
      <descriptorRef>jar-with-dependencies</descriptorRef>
    </descriptorRefs>
  </configuration>
</plugin>
```

```
</configuration>
<executions>
  <execution>
    <id>make-assembly</id>
    <phase>package</phase>
    <goals>
      <goal>single</goal>
    </goals>
  </execution>
</executions>
</plugin>
</plugins>
</build>

</project>
```

1.4.3.2. 写数据到 hudi

数据如下: 将数据写到 hudi

```
1,涛哥,40,M
2,星哥,37,M
3,老娜,29,F
```

```
/**
 * @Date 2021/10/31
 * @Created by HANGGE
 * @Description
 *      将数据写入到 hudi
 */
object Demo01 {
  def main(args: Array[String]): Unit = {
    System.setProperty("HADOOP_USER_NAME", "root");
    val sparkConf = new SparkConf().setAppName("test_hudi")
      .set("spark.serializer", "org.apache.spark.serializer.KryoSerializer")
    //1 获取 sparksession 对象
    val sparkSession = SparkSession.builder().config(sparkConf).getOrCreate()
    // 加载数据 将 DF 写入到 HUDI 中
    // 创建表数据结构
    val schema = new StructType().add("id", DataTypes.StringType).add("name",
      DataTypes.StringType).add("age", DataTypes.IntegerType).add("gender", DataTypes.StringType)
    val commitTime = System.currentTimeMillis().toString //生成提交时间
    //
```

```
// val df: DataFrame =
sparkSession.read.schema(schema).csv("/hudi_data/user.csv").withColumn("ts" , lit(commitTime))

val df: DataFrame = sparkSession.read.schema(schema).csv("/data/teacher.csv").withColumn("ts" ,
lit(commitTime))

df.write.format("hudi")
  .options(getQuickstartWriteConfigs)
  .option(DataSourceWriteOptions.PRECOMBINE_FIELD.key(), "ts") // 提交时间
  .option(DataSourceWriteOptions.RECORDKEY_FIELD.key(), "id") // 主键
  .option(DataSourceWriteOptions.PARTITIONPATH_FIELD.key(), "gender")// 分区字段
  .option(TBL_NAME.key(), "tb_teacher")// 表名
  .mode(Append)
  .save("hdfs://doit01:8020/hudi_wd/tbls/tb_teacher")
}
}
```

如果是 windows 系统的电脑作为开发平台,和 HDFS 上的路径不一致,可能出现路径不匹配问题,我们可以阿静程序打包到虚拟机 在虚拟机上运行程序
打包 ,将打好的包上传到虚拟机上

```
spark-submit --master local --executor-memory 1g --total-executor-cores 4 --class com
.doitedu.demo.Demo01 ./hudi.jar
```

1.4.3.3. 从 hudi 中读取数据

```
package com.doitedu.demo

import org.apache.hudi.DataSourceWriteOptions
import org.apache.hudi.QuickstartUtils.getQuickstartWriteConfigs
import org.apache.hudi.config.HoodieWriteConfig.TBL_NAME
import org.apache.spark.SparkConf
import org.apache.spark.sql.SaveMode.Append
import org.apache.spark.sql.types.{DataTypes, StructType}
import org.apache.spark.sql.{DataFrame, SparkSession}

/**
 * @Date 2021/10/31
 * @Created by HANGGE
 * @Description
 * 加载 hudi 中的数据
 */
object Demo02 {
  def main(args: Array[String]): Unit = {
    System.setProperty("HADOOP_USER_NAME", "root");
```

```
val sparkConf = new SparkConf().setAppName("test_hudi").setMaster("local[*]")
    .set("spark.serializer", "org.apache.spark.serializer.KryoSerializer")
//1 获取 sparksession 对象
val sparkSession = SparkSession.builder().config(sparkConf).getOrCreate()
import org.apache.spark.sql.functions._
// 加载数据 将 DF 写入到 HUDI 中
// 创建表数据结构
val schema = new StructType().add("id", DataTypes.StringType).add("name",
DataTypes.StringType).add("age", DataTypes.IntegerType).add("gender", DataTypes.StringType)
val commitTime = System.currentTimeMillis().toString //生成提交时间

val df: DataFrame =
sparkSession.read.format("hudi").load("hdfs://doit01:8020/hudi_wd/tbls/tb_teacher")
    df.show()
}
```

1.5. flink 读写 Hudi

1.5.1. flink 下载和部署

① 下载 flink 安装包, 下载地址: <https://flink.apache.org/downloads.html>

② 上传 flink 安装包到 Linux 服务器上

③ 解压 flink 安装包

```
tar -xvf flink-1.12.2-bin-scala_2.12.tgz -C /bigdata/
```

④ 修改 conf 目录下的 flink-conf.yaml 配置文件

#指定 jobmanager 的地址

```
jobmanager.rpc.address: doit01
```

#指定 taskmanager 的可用槽位的数量

```
taskmanager.numberOfTaskSlots: 4
```

⑤ 修改 conf 目录下的 workers 配置文件, 指定 taskmanager 的所在节点

```
doit01
```

```
doit02
```

```
doit03
```

⑥ 将配置好的 Flink 拷贝到其他节点

```
for i in {1..3}; do scp -r flink-1.12.2 doit0$i:$PWD; done
```

⑦ 添加 hadoop 的环境变量

```
export HADOOP_COMMON_HOME=/opt/apps/hadoop-2.7.3
```

```
export HADOOP_HDFS_HOME=/opt/apps/hadoop-2.7.3
```



```
export HADOOP_YARN_HOME=/opt/apps/hadoop-2.7.3
export HADOOP_MAPRED_HOME=/opt/apps/hadoop-2.7.3
export HADOOP_CLASSPATH=`hadoop classpath`
```

如果端口被占用可以修改端口号

⑧ 复制 jar 包并分发到集群

将这两个 jar 包复制到 flink 集群所有的节点上

```
hudi-flink-bundle_2.12-0.9.0.jar
hudi-hadoop-mr-bundle-0.9.0.jar
```

⑨ 启动集群

bin/start-cluster.sh

⑩ 配置系统环境变量

vi /etc/profile

⑪ 访问 webUI 页面 默认端口是 8081 如果冲突要修改, 我的是 8888

Path, ID	Data Port	Last Heartbeat	All Slots	Free Slots	CPU Cores	Physical MEM	JVM Heap Size	Flink Managed MEM
192.168.133.99:44886-53783c akka.tcp://Flink@192.168.133.99:44888/user/tpc/taskmanager_0	44051	21-10-31 14:28:19	4	4	4	3.68 GB	512 MB	512 MB
192.168.133.77:40862-59776c akka.tcp://Flink@192.168.133.77:40862/user/tpc/taskmanager_0	37996	21-10-31 14:28:19	4	4	4	3.68 GB	512 MB	512 MB
192.168.133.88:40304-b4b56f akka.tcp://Flink@192.168.133.88:40304/user/tpc/taskmanager_0	37116	21-10-31 14:28:19	4	4	4	3.68 GB	512 MB	512 MB

1.5.2. 启动 flink-sql 客户端

```
sql-client.sh embedded shell
```



```
-- 设置 flinksql 直接显示结果
set execution.result-mode=tableau;
-- 批处理数据
set execution.type = batch;
-- 创建表
```

```
CREATE TABLE t1(
  uuid VARCHAR(20),
  name VARCHAR(10),
  age INT,
  ts TIMESTAMP(3),
  `partition` VARCHAR(20)
)
PARTITIONED BY (`partition`)
WITH (
  'connector' = 'hudi',
  'path' = '/hufi_flink/t1', -- 注意这个路径
  'table.type' = 'MERGE_ON_READ'
);

show tables ;

-- 插入数据
INSERT INTO t1 VALUES
  ('id1', 'Danny', 23, TIMESTAMP '1970-01-01 00:00:01', 'par1'),
  ('id2', 'Stephen', 33, TIMESTAMP '1970-01-01 00:00:02', 'par1'),
  ('id3', 'Julian', 53, TIMESTAMP '1970-01-01 00:00:03', 'par2'),
  ('id4', 'Fabian', 31, TIMESTAMP '1970-01-01 00:00:04', 'par2'),
  ('id5', 'Sophia', 18, TIMESTAMP '1970-01-01 00:00:05', 'par3'),
  ('id6', 'Emma', 20, TIMESTAMP '1970-01-01 00:00:06', 'par3'),
  ('id7', 'Bob', 44, TIMESTAMP '1970-01-01 00:00:07', 'par4'),
  ('id8', 'Han', 56, TIMESTAMP '1970-01-01 00:00:08', 'par4');

-- 查询数据
select * from t1 ;

+-----+-----+-----+-----+-----+
| uuid |   name | age |          ts | partition |
+-----+-----+-----+-----+-----+
| id4 | Fabian | 31 | 1970-01-01T00:00:04 | par2 |
| id3 | Julian | 53 | 1970-01-01T00:00:03 | par2 |
| id8 | Han | 56 | 1970-01-01T00:00:08 | par4 |
| id7 | Bob | 44 | 1970-01-01T00:00:07 | par4 |
| id6 | Emma | 20 | 1970-01-01T00:00:06 | par3 |
| id5 | Sophia | 18 | 1970-01-01T00:00:05 | par3 |
| id2 | Stephen | 33 | 1970-01-01T00:00:02 | par1 |
| id1 | Danny | 23 | 1970-01-01T00:00:01 | par1 |
+-----+-----+-----+-----+-----+
```

/hufi_flink/t1							Go!
Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
drwxr-xr-x	root	supergroup	0 B	2021/11/4 上午12:54:59	0	0 B	.hoodie
drwxr-xr-x	root	supergroup	0 B	2021/11/4 上午12:54:59	0	0 B	par1
drwxr-xr-x	root	supergroup	0 B	2021/11/4 上午12:54:59	0	0 B	par2
drwxr-xr-x	root	supergroup	0 B	2021/11/4 上午12:54:59	0	0 B	par3
drwxr-xr-x	root	supergroup	0 B	2021/11/4 上午12:54:31	0	0 B	par4

1.5.3. 更新数据

分区内主键相同的数据会被覆盖掉, 于是实现数据的更新操作

```
insert into t1 values
('id3','HANGGE',27,TIMESTAMP '1999-01-01 00:00:01','par4');
```

```
Flink SQL> select * from t1 ;
+-----+-----+-----+-----+-----+
| uuid | name | age | ts | partition |
+-----+-----+-----+-----+
| id3 | ADA | 23 | 1990-01-01T00:00:01 | par4 |
| id8 | Han | 56 | 1970-01-01T00:00:08 | par4 |
| id7 | Bob | 44 | 1970-01-01T00:00:07 | par4 |
| id4 | Fabian | 31 | 1970-01-01T00:00:04 | par2 |
| id3 | Julian | 53 | 1970-01-01T00:00:03 | par2 |
| id6 | Emma | 20 | 1970-01-01T00:00:06 | par3 |
| id5 | Sophia | 18 | 1970-01-01T00:00:05 | par3 |
| id2 | Stephen | 33 | 1970-01-01T00:00:02 | par1 |
| id1 | Danny | 27 | 1970-01-01T00:00:01 | par1 |
+-----+-----+-----+-----+
9 rows in set

Flink SQL> insert into t1 values
> ('id3','HANGGE',27,TIMESTAMP '1999-01-01 00:00:01','par4');
[INFO] Submitting SQL update statement to the cluster...
[INFO] Table update statement has been successfully submitted to the cluster:
Job ID: 818994457ae4ed8297f4a0467ca0d965

Flink SQL> select * from t1 ;
+-----+-----+-----+-----+-----+
| uuid | name | age | ts | partition |
+-----+-----+-----+-----+
| id6 | Emma | 20 | 1970-01-01T00:00:06 | par3 |
| id5 | Sophia | 18 | 1970-01-01T00:00:05 | par3 |
| id4 | Fabian | 31 | 1970-01-01T00:00:04 | par2 |
| id3 | Julian | 53 | 1970-01-01T00:00:03 | par2 |
| id3 | HANGGE | 27 | 1999-01-01T00:00:01 | par4 |
| id8 | Han | 56 | 1970-01-01T00:00:08 | par4 |
| id7 | Bob | 44 | 1970-01-01T00:00:07 | par4 |
| id2 | Stephen | 33 | 1970-01-01T00:00:02 | par1 |
| id1 | Danny | 27 | 1970-01-01T00:00:01 | par1 |
+-----+-----+-----+-----+
9 rows in set
```

1.5.4. 流式查询(实时)

展示出 read.streaming.start-commit 提交后发生的所有更改 ,此功能的独特之处在于它现在允许您在流式或批处理数据源上创作流式管道。{: .notice--info}

开启两个 SQL 客户端窗口, 一个写操作 ,一个读操作
窗口一

```
CREATE TABLE t1(  
  uuid VARCHAR(20),  
  name VARCHAR(10),  
  age INT,  
  ts TIMESTAMP(3),  
  `partition` VARCHAR(20)  
)  
PARTITIONED BY (`partition`)  
WITH (  
  'connector' = 'hudi',  
  'path' = '/hufi_flink/t1',  
  'table.type' = 'MERGE_ON_READ',  
  'read.streaming.enabled' = 'true', -- this option enable the streaming read  
  'read.streaming.start-commit' = '20210316134557' , -- specifies the start  
commit instant time  
  'read.streaming.check-interval' = '10' -- specifies the check interval for  
finding new source commits, default 60s.  
);  
-- 实时查询  
select * from t1 ;
```

窗口二

```
set execution.result-mode=tableau;  
SET execution.type = batch;  
  
CREATE TABLE t1(  
  uuid VARCHAR(20),  
  name VARCHAR(10),  
  age INT,  
  ts TIMESTAMP(3),  
  `partition` VARCHAR(20)  
)  
PARTITIONED BY (`partition`)  
WITH (  
  'connector' = 'hudi',  
  'path' = '/hufi_flink/t1',  
  'table.type' = 'MERGE_ON_READ'  
);  
-- 插入数据  
INSERT INTO t1 VALUES  
  ('id9','9999',23,TIMESTAMP '2021-11-04 03:00:01','par4') ;
```

在窗口一种观察数据现象,数据会不断的展现在视图中

展示出 `read.streaming.start-commit` 提交后发生的所有更改,此功能的独特之处在于它现在允许您在流式或批处理数据源上创作流式管道。{: .notice--info}

uuid	name	age	ts	partition
id6	Emma	20	1970-01-01T00:00:06	par3
id5	Sophia	18	1970-01-01T00:00:05	par3
id2	Stephen	33	1970-01-01T00:00:02	par1
id1	Danny	27	1970-01-01T00:00:01	par1
id4	Fabian	31	1970-01-01T00:00:04	par2
id3	Julian	53	1970-01-01T00:00:03	par2
id3	HANGGE	27	1999-01-01T00:00:01	par4
id8	Han	56	1970-01-01T00:00:08	par4
id7	Bob	44	1970-01-01T00:00:07	par4
id9	9999	23	2021-11-04T03:00:01	par4
id9	9999	23	2021-11-04T03:00:01	par9
id9	9999	23	2021-11-04T03:00:01	par10
id9	9999	23	2021-11-04T03:00:01	par11

1.6. spark 操作 hudi 详解

1.6.1. 更新数据

数据的保存模式是 **Append** 模式,每次插入新数据,都会使数据生成器生成对现有数据行程的更新!一般基本的更新操作原则是分区内主键相同的数据会被更新!

- 1) 加载 HDFS 上的数据,新数据
- 2) 写入到 hudi Append
- 3) 相同的主键 分区
- 4) `PRECOMBINE_FIELD.key()` 判断数据新旧的依据 一般指定时间

```
package com.doitedu.demo.introduction

import org.apache.hudi.DataSourceWriteOptions
import org.apache.hudi.QuickstartUtils.getQuickstartWriteConfigs
import org.apache.hudi.config.HoodieWriteConfig.TBL_NAME
import org.apache.spark.SparkConf
import org.apache.spark.sql.SaveMode.Append
import org.apache.spark.sql.{DataFrame, SparkSession}
import org.apache.spark.sql.types.{DataTypes, StructType}

/**
 * @Date 2021/10/31
 * @Created by HANGGE
 */
```

```

* @Description
*      更新数据
*      加载 DF 数据 将 DF 数据写入到 hudi 中
*      当主键分区内主键相同的数据会被更新
*/
object UpdateData {

  def main(args: Array[String]): Unit = {
    System.setProperty("HADOOP_USER_NAME", "root");
    val sparkConf = new SparkConf().setAppName("test_hudi")
      .set("spark.serializer", "org.apache.spark.serializer.KryoSerializer")
    //1 获取 sparksession 对象
    val sparkSession = SparkSession.builder().config(sparkConf).getOrCreate()
    import org.apache.spark.sql.functions._
    val schema = new StructType().add("id", DataTypes.StringType).add("name",
DataTypes.StringType).add("age", DataTypes.IntegerType).add("gender",
DataTypes.StringType)
    val commitTime = System.currentTimeMillis().toString //生成提交时间

    val df: DataFrame =
sparkSession.read.schema(schema).csv("/data/teacher2.csv").withColumn("ts" ,
lit(commitTime))

    df.write.format("hudi")
      .options(getQuickstartWriteConfigs)
      .option(DataSourceWriteOptions.PRECOMBINE_FIELD.key(), "ts") // 提交时间
      .option(DataSourceWriteOptions.RECORDKEY_FIELD.key(), "id") // 主键
      .option(DataSourceWriteOptions.PARTITIONPATH_FIELD.key(), "gender")// 分区字
段
      .option(TBL_NAME.key(), "tb_teacher")// 表名
      .mode(Append)
      .save("/hudi_wd/tbls/tb_teacher")
    sparkSession.close()
  }
}

```

更新后.查询数据 ,发现数据发生了变化

_hoodie_commit_time _hoodie_commit_seqno _hoodie_record_key _hoodie_partition_path _hoodie_file_name id name age ts gender
20211101031139 20211101031139_1_2 2 M bdd3a3b1-3a5e-42c... 2 星哥 37 1635707496223 M
20211101031139 20211101031139_1_3 1 M bdd3a3b1-3a5e-42c... 1 涛哥 40 1635707496223 M
20211101031139 20211101031139_0_1 3 F a33151f5-4e06-4d0... 3 NAJIE 33 1635707496223 F

1.6.2. 增量查询之时间点查询

在 hudi 中 ,将用户的每次提交都记录在时间轴线上 , 维护一个递增的提交时间 ,每次提交时间的数据 , 以增量的方式保存 ,方便我们快速的增量查询 !

指定点 : -----> 增量 1 增量 2...

指定时间段:

```
package com.doitedu.demo.introduction

import com.doitedu.demo.utils.SparkUtil
import org.apache.hudi.{DataSourceReadOptions, DataSourceWriteOptions}
import org.apache.log4j.{Level, Logger}
import org.apache.spark.sql.{DataFrame, SparkSession}
/**
 * @Date 2021/10/31
 * @Created by HANGGE
 * @Description
 *          按照时间点查询数据
 */
object C02_QueryPointInTime {
  Logger.getLogger("org").setLevel(Level.ERROR)
  def main(args: Array[String]): Unit = {
    val session: SparkSession = SparkUtil.getSparkSession
    // 加载表中数据
    val df: DataFrame =
      session.read.format("hudi").load("hdfs://doit01:8020/hudi_wd/tb1s/tb_teacher")
    // 创建视图展示最新数据
    df.createOrReplaceTempView("tb_teacher")
    import org.apache.spark.sql.functions._
    import session.implicitly._
    // 获取最近几次提交的时间
    val df2: DataFrame = session.sql("select distinct(_hoodie_commit_time)
as commitTime from tb_teacher order by commitTime")
    // 最后一批导入的数据
    val arr: Array[String] = df2.map(row =>
      row.getAs[String]("commitTime")).take(10)
    val beginTime: String = arr(arr.length - 2)

    // 最后两次批导入的数据
    // val beginTime: String = arr(arr.length - 3)

    val res = session.read.format("hudi").
      option(DataSourceReadOptions.QUERY_TYPE.key(),
        DataSourceReadOptions.QUERY_TYPE_INCREMENTAL_OPT_VAL).
      option(DataSourceReadOptions.BEGIN_INSTANTTIME.key(), beginTime).
      load("hdfs://doit01:8020/hudi_wd/tb1s/tb_teacher")
  }
}
```

```
res.createOrReplaceTempView("tb_teacher_res")
session.sql("select * from tb_teacher_res").show()

session.close()
}
}
```

原始数据-->更新数据-->导入数据(1)-->导入数据(2)

获取提交的时间总共为 3 次

使用 spark-SQL 更新数据

1.6.3. 增量查询之时间范围查询

```
package com.doitedu.demo.introduction

import com.doitedu.demo.utils.SparkUtil
import org.apache.hudi.DataSourceReadOptions
import org.apache.log4j.{Level, Logger}
import org.apache.spark.sql.{DataFrame, SparkSession}

/**
 * @Date 2021/10/31
 * @Created by HANGGE
 * @Description
 *          按照时间点查询数据
 */
object C02_QueryRangeTime {
  Logger.getLogger("org").setLevel(Level.ERROR)
  def main(args: Array[String]): Unit = {
    val session: SparkSession = SparkUtil.getSparkSession
    // 加载表中数据
    val df: DataFrame =
      session.read.format("hudi").load("hdfs://doit01:8020/hudi_wd/tbls/tb_teacher")
    // 创建视图展示最新数据
    df.createOrReplaceTempView("tb_teacher")
    import session.implicits._
    // 获取最近几次提交的时间
    val df2: DataFrame = session.sql("select distinct(_hoodie_commit_time) as
      commitTime from tb_teacher order by commitTime")
  }
}
```



```
// 最后一批导入的数据
val arr: Array[String] = df2.map(row => row.getAs[String]("commitTime")).take(10)
// 最后一次提交 这个起始时间可以自己设定
val beginTime: String = arr(arr.length - 3)
// =倒数第二次提交
val endTime: String = arr(arr.length - 2)

val res = session.read.format("hudi").
  option(DataSourceReadOptions.QUERY_TYPE.key(),
DataSourceReadOptions.QUERY_TYPE_INCREMENTAL_OPT_VAL)
  .option(DataSourceReadOptions.BEGIN_INSTANTTIME.key(), beginTime)
  .option(DataSourceReadOptions.END_INSTANTTIME.key(), endTime)
  .load("hdfs://doit01:8020/hudi_wd/tb1s/tb_teacher")
res.createOrReplaceTempView("tb_teacher_res")
session.sql("select * from tb_teacher_res").show()
session.close()
}
}
```

_hoodie_commit_time _hoodie_commit_seqno _hoodie_record_key _hoodie_partition_path _hoodie_file_name id name age gender ts
20211101033655 20211101033655_1_2 4 M bdd3a3b1-3a5e-42c... 4 王丽坤 40 M 1635709012336
20211101033655 20211101033655_1_3 5 M bdd3a3b1-3a5e-42c... 5 江疏影 37 M 1635709012336
20211101033655 20211101033655_0_1 6 F a33151f5-4e06-4d0... 6 高圆圆 29 F 1635709012336

1.6.4. 删除数据

```
package com.doitedu.demo.introduction

import com.doitedu.demo.utils.SparkUtil
import org.apache.hudi.DataSourceWriteOptions
import org.apache.spark.sql.{DataFrame, SparkSession}
import org.apache.spark.sql.types.{DataTypes, StructType}
import org.apache.hudi.QuickstartUtils.getQuickstartWriteConfigs
import org.apache.spark.sql.SaveMode.Append
import org.apache.hudi.config.HoodieWriteConfig.TBL_NAME

/**
 * @Date 2021/11/1
 * @Created by HANGGE
 * @Description
 */
object C03_DelateData {
  def main(args: Array[String]): Unit = {
    import org.apache.spark.sql.functions._
  }
}
```

```

    val sparkSession: SparkSession = SparkUtil.getSparkSession
    // val schema: StructType = new StructType().add("id", DataTypes.StringType).add("name",
    DataTypes.StringType).add("age", DataTypes.IntegerType).add("gender", DataTypes.StringType)
    // 加载所有数据
    val df: DataFrame =
    sparkSession.read.format("hudi").load("hdfs://doit01:8020/hudi_wd/tbls/tb_teacher")
    df.createOrReplaceTempView("tb_teacher")
    val commitTime = System.currentTimeMillis().toString //生成提交时间
    val deleteDF: DataFrame = sparkSession.sql("select * from tb_teacher where id =
    8").withColumn("ts", lit(commitTime))
    deleteDF.show()
    deleteDF
    .write
    .format("hudi")
    .options(getQuickstartWriteConfigs())
    .option(DataSourceWriteOptions.OPERATION.key(), "delete")
    .option(DataSourceWriteOptions.PRECOMBINE_FIELD.key(), "ts")
    .option(DataSourceWriteOptions.RECORDKEY_FIELD.key(), "id")
    .option(DataSourceWriteOptions.PARTITIONPATH_FIELD.key(), "gender")// 分区字段
    .option(TBL_NAME.key(), "tb_teacher")
    .mode(Append)
    .save("hdfs://doit01:8020/hudi_wd/tbls/tb_teacher")
    sparkSession.close()
  }
}

```

将程序打包执行

```
spark-submit --master local --executor-memory 1g --total-executor-cores 4 --class
com.doitedu.demo.introduction.C03_DeleteData
```

查询数据发现数据被删除 少了 id 为 8 的数据

_hoodie_commit_time	_hoodie_commit_seqno	_hoodie_record_key	_hoodie_partition_path	_hoodie_file_name	id	name	age	ts	gender
20211101031139	20211101031139_1_2	2	M bdd3a3b1-3a5e-42c...	2 星哥	37	1635707496223	M		
20211101031139	20211101031139_1_3	1	M bdd3a3b1-3a5e-42c...	1 涛哥	40	1635707496223	M		
20211101033655	20211101033655_1_2	4	M bdd3a3b1-3a5e-42c...	4 王丽坤	40	1635709012336	M		
20211101033655	20211101033655_1_3	5	M bdd3a3b1-3a5e-42c...	5 江疏影	37	1635709012336	M		
20211101033725	20211101033725_1_2	7	M bdd3a3b1-3a5e-42c...	7 黄渤	40	1635709042253	M		
20211101031139	20211101031139_0_1	3	F a33151f5-4e06-4d0...	3 NAJIE	33	1635707496223	F		
20211101033655	20211101033655_0_1	6	F a33151f5-4e06-4d0...	6 高圆圆	29	1635709012336	F		
20211101033725	20211101033725_0_1	9	F a33151f5-4e06-4d0...	9 孙红雷	29	1635709042253	F		

启动 spark-sql 窗口

```
spark-sql --packages org.apache.hudi:hudi-spark3-bundle_2.12:0.9.0,org.apache.spark:spark-avro_2.12:3.0.1 --conf
'spark.serializer=org.apache.spar
k.serializer.KryoSerializer' --conf 'spark.sql.extensions=org.apache.spark.sql.hudi.HoodieSparkSessionExtension'
```

根据 hudi 目录创建表

```
create table tb_user2 using hudi location '/hudi_dw/tb_user2';
```

```
create table tb_user2 using hudi location '/hudi_dw/tb_user2' options (  
  type = 'cow',  
  primaryKey = 'id',  
  preCombineField = 'ts'  
)  
partitioned by (city) ;
```

delete from where 删除数据

1.6.5. 修改数据结构

1.6.6. spark-sql 指令

对 SQL 的支持还不是很完整

1) 在已存在的表目录上创建表

```
create table tb_teacher using hudi location 'hdfs://doit01:8020/hudi_dw/tbls/tb_teacher' ;
```

80211101031139	20211101031139_1_2	2	M	bdd3a3b1-3a5e-42c8-8d88-5e779a6efa3e-0_1-26-28_20211101031139.parquet	2	星哥	37	M
835707496223	20211101031139_1_3	1	M	bdd3a3b1-3a5e-42c8-8d88-5e779a6efa3e-0_1-26-28_20211101031139.parquet	1	涛哥	40	M
80211101031139	20211101031139_1_3	1	M	bdd3a3b1-3a5e-42c8-8d88-5e779a6efa3e-0_1-26-28_20211101031139.parquet	1	涛哥	40	M
835707496223	20211101031139_1_3	1	M	bdd3a3b1-3a5e-42c8-8d88-5e779a6efa3e-0_1-26-28_20211101031139.parquet	1	涛哥	40	M
80211101033655	20211101033655_1_2	4	M	bdd3a3b1-3a5e-42c8-8d88-5e779a6efa3e-0_1-26-24_20211101033655.parquet	4	王丽坤	40	M
835709012336	20211101033655_1_3	5	M	bdd3a3b1-3a5e-42c8-8d88-5e779a6efa3e-0_1-26-24_20211101033655.parquet	5	江疏影	37	M
80211101033655	20211101033655_1_3	5	M	bdd3a3b1-3a5e-42c8-8d88-5e779a6efa3e-0_1-26-24_20211101033655.parquet	5	江疏影	37	M
835709012336	20211101033655_1_3	5	M	bdd3a3b1-3a5e-42c8-8d88-5e779a6efa3e-0_1-26-24_20211101033655.parquet	5	江疏影	37	M
80211101033725	20211101033725_1_2	7	M	bdd3a3b1-3a5e-42c8-8d88-5e779a6efa3e-0_1-26-24_20211101033725.parquet	7	黄渤	40	M
835709042253	20211101033725_1_2	7	M	bdd3a3b1-3a5e-42c8-8d88-5e779a6efa3e-0_1-26-24_20211101033725.parquet	7	黄渤	40	M
80211101031139	20211101031139_0_1	3	F	a33151f5-4e06-4d09-b0c0-d915d01e0681-0_0-20-27_20211101031139.parquet	3	NAJIE	33	F
835707496223	20211101031139_0_1	3	F	a33151f5-4e06-4d09-b0c0-d915d01e0681-0_0-20-27_20211101031139.parquet	3	NAJIE	33	F
80211101033655	20211101033655_0_1	6	F	a33151f5-4e06-4d09-b0c0-d915d01e0681-0_0-20-23_20211101033655.parquet	6	高圆圆	29	F
835709012336	20211101033655_0_1	6	F	a33151f5-4e06-4d09-b0c0-d915d01e0681-0_0-20-23_20211101033655.parquet	6	高圆圆	29	F
80211101033725	20211101033725_0_1	9	F	a33151f5-4e06-4d09-b0c0-d915d01e0681-0_0-20-23_20211101033725.parquet	9	孙红雷	29	F
835709042253	20211101033725_0_1	9	F	a33151f5-4e06-4d09-b0c0-d915d01e0681-0_0-20-23_20211101033725.parquet	9	孙红雷	29	F

2) 修改表名

```
alter table tb_teacher rename to my_teacher ;
```

3) 修改表结构 添加字段

```
alter table my_teacher add column (address string) ;
```

_hoodie_commit_time	string	NULL
_hoodie_commit_segno	string	NULL
_hoodie_record_key	string	NULL
_hoodie_partition_path	string	NULL
_hoodie_file_name	string	NULL
id	string	NULL
name	string	NULL
age	int	NULL
gender	string	NULL
ts	string	NULL
address	string	NULL

4) 修改字段的数据类型

```
alter table my_teacher change column age age bigint ;
```

第二篇 Hudi 原理深入

hudi 可以以数据流的方式更新和删除 hadoop 中的数据 , 并且记录数据的更新和删除, 从而大大提升增量 , 批量获取数据的能力!

1.7. 时间轴

用户在表上的所有操作都会记录操作的时间 , 这有助于提供表的即时视图, 同时还有效地支持按到达顺序检索数据,

Hudi 的时刻由以下组件组成:

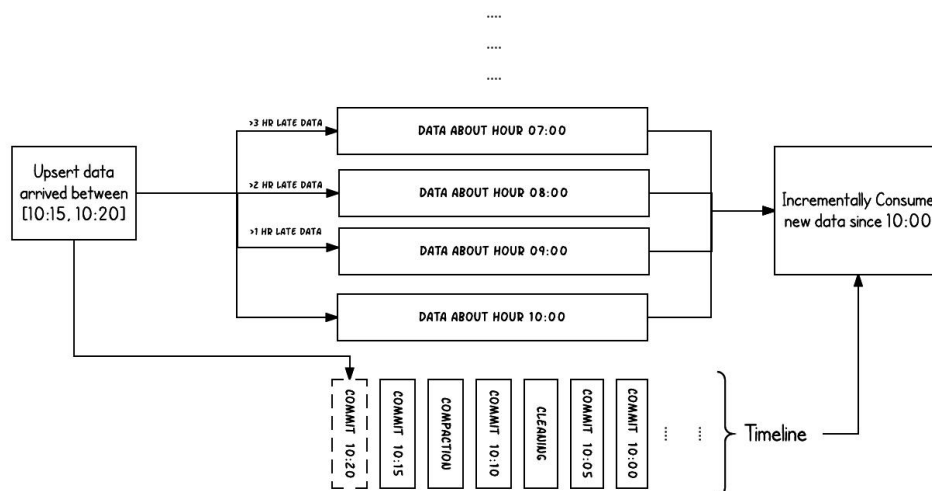
- (1) Instant action: 在表上执行的操作类型
- (2) Instant time: 即时时间, 通常是一个时间戳, 它安装 action 的开始时间单调递增
- (3) State: 时刻的当前状态

记录了用户操作的类型, 操作的时间, 操作的状态!

Hudi 保证在时间线上执行的操作是基于即时时间的原子和时间线一致。每次操作记录操作的时间, 在这个操作时间快照中数据保证原子性!

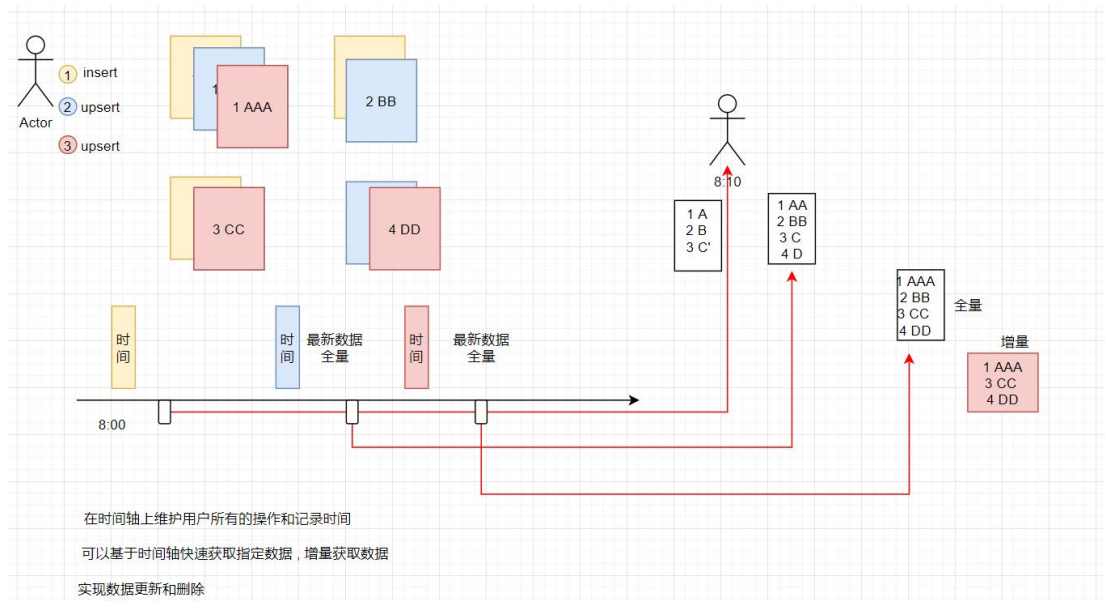
- **COMMIT**- 提交表示将一批记录原子写入表中。
- **CLEAN**- 清除表中不再需要的旧版本文件的后台活动。
- **DELTA_COMMIT**- 增量提交是指将一批记录原子写入 MergeOnRead 类型表中, 其中部分/所有数据可以仅写入增量日志。
- **COMPACTION**- 协调 Hudi 中的差异数据结构的后台活动, 例如: 将更新从基于行的日志文件移动到列格式。在内部, 压缩表现为时间线上的特殊提交
- **ROLLBACK** - 表示提交/增量提交不成功并回滚, 删除在此类写入过程中产生的任何部分文件
- **SAVEPOINT**- 将某些文件组标记为“已保存”, 这样清洁工就不会删除它们。在发生灾难/数据恢复情况时, 它有助于将表恢复到时间线上的某个点。

任何一个时间点的状态一定是, 操作已经请求, 操作在处理, 操作已经处理完毕



图显示了 hudi 表上 10:00 和 10:20 之间发生的更新插入, 每 5 分钟一次, 将提交元数据留以及其他后台清理/压缩操作在 hudi 时间轴上。观察的关键点是, 提交时间表示数据的到达时间, 而实际数据组织则反应了实际时间或事件时间, 即数据所反映的从 07:00 开始的每小时时段。在权衡数据延迟和完整性, 这是两个关键概念。

如果有延迟到达的数据 (事件时间为 9:00 的数据在 10:20 达到, 延迟 > 1 小时), 可以看到 upsert 将数据生成到更旧的时间段/文件夹中。在时间轴的帮助下, 增量查询可以只提取 10:00 以后成功提交的新数据, 并非高效地只消费更改过的文件, 且无需扫描更大的文件范围, 例如 07:00 后的所有时间段。

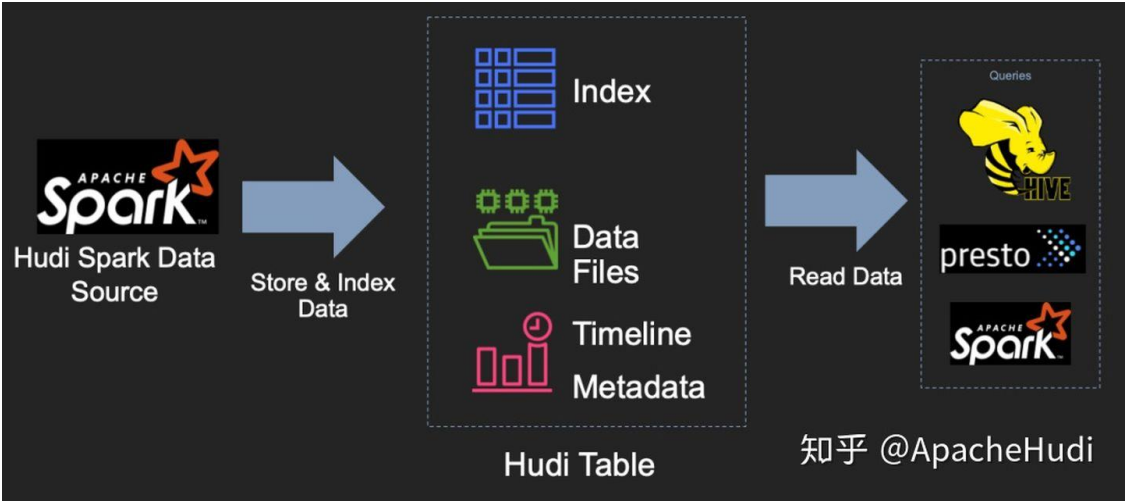


1.8. 存储原理

Hudibasepath 在 DFS 下将一个表组织成一个目录结构。表被分成多个分区, 这些分区是包含该分区的数据文件的文件夹, 与 Hive 表非常相似。每个分区都由它的 唯一标识 partitionpath, 它是相对于基本路径的。

在每个分区内, 文件被组织成 file groups, 由 file id. 每个文件组包含多个 file slices, 其中每个切片包含 *.parquet 在某个提交/压缩即时时间生成的基本文件 (), 以及 *.log.* 包含自生成基本文件以来对基本文件的插入/更新的一组日志文件 ()。Hudi 采用 MVCC 设计, 其中压缩操作合并日志和基础文件以生成新的文件切片, 清理操作清除未使用/较旧的文件切片以回收 DFS 上的空间。

现在让我们更深入地研究 Hudi 系统本身的设计。在较高的层次上, 用于写 Hudi 表的组件使用了一种受支持的方式嵌入到 Apache Spark 作业中, 它会在支持 DFS 的存储上生成代表 Hudi 表的一组文件。然后, 在具有一定保证的情况下, 诸如 Apache Spark、Presto、Apache Hive 之类的查询引擎可以查询该表。Hudi 表的三个主要组件: 1) 有序的时间轴元数据。类似于数据库事务日志。2) 分层布局的数据文件: 实际写入表中的数据。3) 索引 (多种实现方式): 映射包含指定记录的数据集。



/hudi_wd/tbls/tb_teacher								Go!
Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name	
drwxr-xr-x	root	supergroup	0 B	2021/11/1 下午1:47:59	0	0 B	.hoodie	
drwxr-xr-x	root	supergroup	0 B	2021/11/1 下午1:00:02	0	0 B	.hoodie\temp	
drwxr-xr-x	root	supergroup	0 B	2021/11/1 上午3:37:33	0	0 B	F	
drwxr-xr-x	root	supergroup	0 B	2021/11/1 下午1:17:39	0	0 B	M	
drwxr-xr-x	root	supergroup	0 B	2021/11/1 下午12:37:17	0	0 B	default	

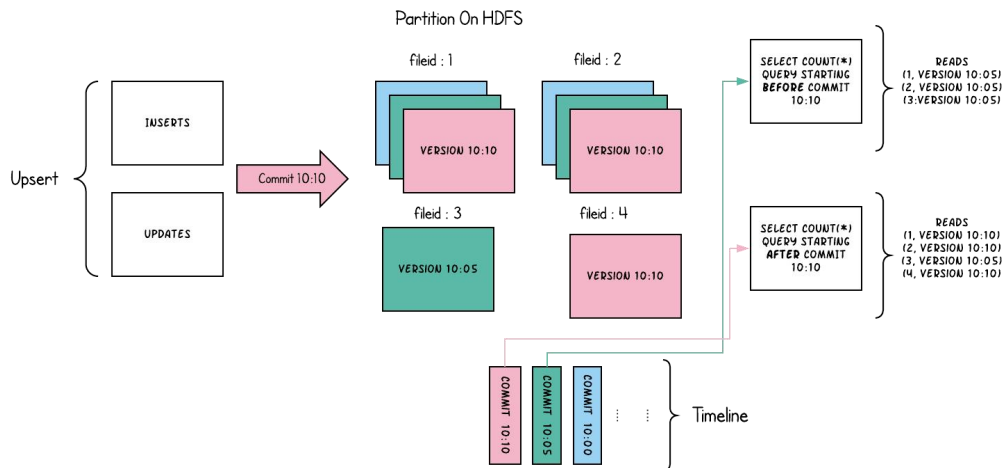
/hudi_wd/tbls/tb_teacher/M								
----------------------------	--	--	--	--	--	--	--	--

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
-rw-r--r--	root	supergroup	93 B	2021/11/1 上午2:49:35	3	128 MB	.hoodie_partition_metadata
-rw-r--r--	root	supergroup	424.66 KB	2021/11/1 下午1:17:39	3	128 MB	bdd3a3b1-3a5e-42c8-8d88-5e779a6efa3e-0_0-38-47_20211101131734.parquet
-rw-r--r--	root	supergroup	424.89 KB	2021/11/1 上午2:49:35	3	128 MB	bdd3a3b1-3a5e-42c8-8d88-5e779a6efa3e-0_1-26-23_20211101024927.parquet
-rw-r--r--	root	supergroup	424.63 KB	2021/11/1 上午3:37:03	3	128 MB	bdd3a3b1-3a5e-42c8-8d88-5e779a6efa3e-0_1-26-24_20211101033655.parquet
-rw-r--r--	root	supergroup	424.67 KB	2021/11/1 上午3:37:34	3	128 MB	bdd3a3b1-3a5e-42c8-8d88-5e779a6efa3e-0_1-26-24_20211101033725.parquet
-rw-r--r--	root	supergroup	424.89 KB	2021/11/1 上午3:11:47	3	128 MB	bdd3a3b1-3a5e-42c8-8d88-5e779a6efa3e-0_1-26-28_20211101031139.parquet

Browse Directory

/hudi_wd/tbls/tb_teacher/.hoodie								Go!
Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name	
drwxr-xr-x	root	supergroup	0 B	2021/11/1 上午2:49:28	0	0 B	.aux	
drwxr-xr-x	root	supergroup	0 B	2021/11/1 下午1:17:39	0	0 B	.temp	
-rw-r--r--	root	supergroup	2.63 KB	2021/11/1 上午2:49:35	3	128 MB	20211101024927.commit	
-rw-r--r--	root	supergroup	0 B	2021/11/1 上午2:49:30	3	128 MB	20211101024927.commit.requested	
-rw-r--r--	root	supergroup	1.76 KB	2021/11/1 上午2:49:33	3	128 MB	20211101024927.inflight	
-rw-r--r--	root	supergroup	2.65 KB	2021/11/1 上午3:11:47	3	128 MB	20211101031139.commit	

数据文件



随着数据被写入，对现有文件组的更新会为该文件组生成一个带有提交即时时间标记的新片，而插入会分配一个新文件组并为该文件组写入它的第一个片。这些文件切片及其提交即时时间在上面用颜色编码。针对此类表运行的 SQL 查询（例如：`select count(*)` 计算该分区中的总记录数），首先检查最新提交的时间线并过滤每个文件组中除最新文件之外的所有文件片。如您所见，旧查询不会看到当前进行中提交的文件颜色编码为粉红色，但在提交后开始的新查询会获取新数据。因此，查询不受任何写入失败/部分写入的影响，并且仅在已提交的数据上运行。

在写表上复制的目的是通过从根本上改进当今表的管理方式

- 一流的支持在文件级自动更新数据，而不是重写整个表/分区
- 能够增量消耗更改，而不是浪费扫描或摸索启发式方法
- 严格控制文件大小以保持出色的查询性能（小文件会显着降低查询性能）。

1.9. 索引

Hudi 通过索引机制将给定的连帽衫键（记录键 + 分区路径）一致地映射到文件 ID，从而提供高效的更新插入。一旦记录的第一个版本写入文件，记录键和文件组/文件 ID 之间的这种映射永远不会改变。简而言之，映射文件组包含一组记录的所有版本。

根据用户查询的数据,找到对应的数据提交版本，定位到具体的文件 ID,加载指定的文件中的数据!

Apache Hudi 使用索引来定位更新/删除所属的文件组。对于 Copy-On-Write 表，通过避免需要连接整个数据集来确定要重写哪些文件，这可以实现快速的 upsert/delete 操作。对于 Merge-On-Read 表，这种设计允许 Hudi 限制任何给定基本文件需要合并的记录数量。具体来说，给定的基本文件只需要针对属于该基本文件一部分的记录的更新进行合并。相比之下，没有索引组件的设计（例如：Apache Hive ACID）可能最终必须根据所有传入的更新/删除记录合并所有基本文件。

从高层次来说，索引将 record key + 可选分区路径映射为一个文件组 ID 并进行存储。而在写操作的时候，我们使用这个映射来路由即将到来的 update/delete 操作到附加到基础文件（MOR）的日志文件，或者需要被合并的 COW 表的最新基础文件。该索引还使 Hudi 能够

根据记录键强制执行唯一约束。

- **Bloom Index (default)** 使用根据记录键构建的 bloom 过滤器, 也可以使用记录键范围修剪候选文件。(更多布隆过滤器的知识可参考文末列出的文章)
- **Simple Index** 根据从存储表中提取的键对传入的更新/删除记录执行精简连接
- **HBase Index** 将索引映射存储在外部 hbase 表中

用户可以使用 `hoodie.index.type` 配置选项选择这些选项之一。此外, 还可以使用 `hoodie.index.class` 并提供 `SparkHoodieIndex` 的子类(适用于 `Apache Spark` 编写者)来使用自定义索引实现

另一个值得理解的关键方面是全局索引和非全局索引之间的区别。布隆和简单索引都有全局选项 - `hoodie.index.type=GLOBAL_BLOOM` 和 `hoodie.index.type=GLOBAL_SIMPLE`。HBase 索引本质上是一个全局索引。

- **全局索引:** 全局索引强制跨表的所有分区的键的唯一性, 即保证表中对于给定的记录键恰好存在一条记录。全局索引提供了更强的保证, 但更新/删除成本随着表 $O(\text{表大小})$ 的大小而增长, 这对于较小的表可能仍然是可以接受的。
- **非全局索引:** 另一方面, 默认索引实现仅在特定分区内强制执行此约束。可以想象, 非全局索引依赖于编写器在更新/删除期间为给定的记录键提供相同的一致分区路径, 但可以提供更好的性能, 因为索引查找操作变为 $O(\text{更新/删除的记录数})$ 并且可以很好地扩展写入量。

1. 10. 表类型

Table Type	Supported Query types
Copy On Write	Snapshot Queries + Incremental Queries
Merge On Read	Snapshot Queries + Incremental Queries + Read Optimized Queries

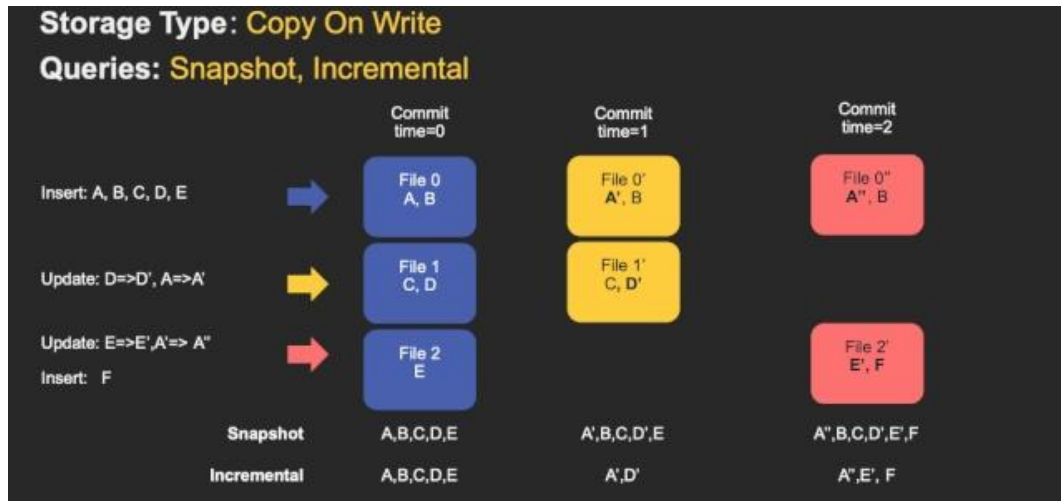
不同的表类型, 决定数据写入和数据读取的方式不同, 从而上层的操作性能上是有一定的差距的!

Trade-off	CopyOnWrite	MergeOnRead
Data Latency	Higher	Lower
Query Latency	Lower	Higher
Update cost (I/O)	Higher (rewrite entire parquet)	Lower (append to delta log)
Parquet File Size	Smaller (high update(I/O) cost)	Larger (low update cost)
Write Amplification	Higher	Lower (depending on compaction strategy)

- **Snapshot Queries:** 查询看到表在给定提交或压缩操作时的最新快照。
在读取表上合并的情况下,它通过即时合并最新文件切片的基本文件和增量文件来公开近乎实时的数据(几分钟)。对于写表上的复制,它提供了对现有镶木地板表的直接替换,同时提供了 **upsert/delete** 和其他写端功能。
- **增量查询:** 查询只看到写入表的新数据,因为给定的提交/压缩。这有效地提供了变更流以启用增量数据管道。
- **Read Optimized Queries:** 查询查看给定提交/压缩操作时表的最新快照。仅公开最新文件切片中的基本/列文件,并保证与非 Hudi 列式表相比具有相同的列式查询性能。

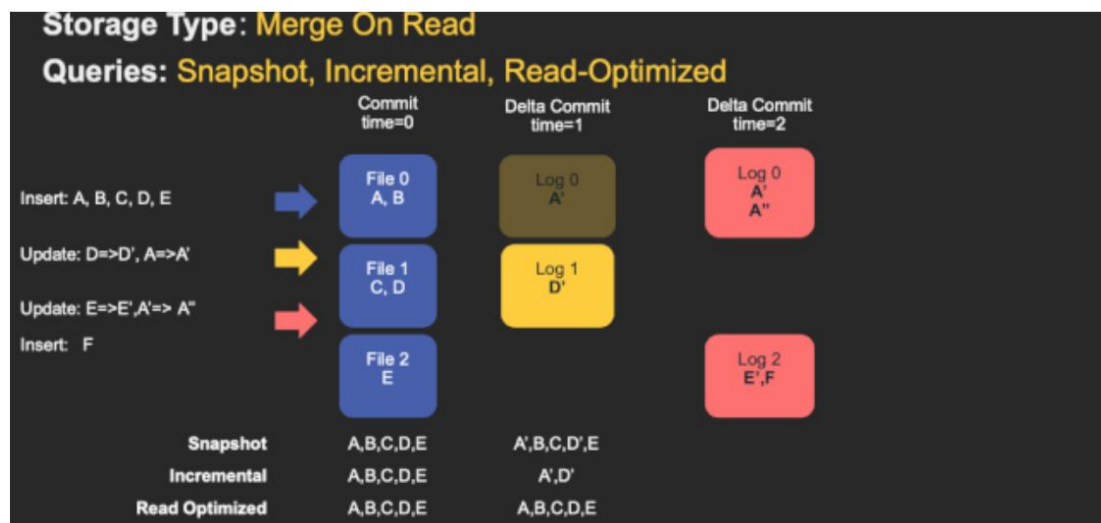
1.10.1. Copy On Write 表

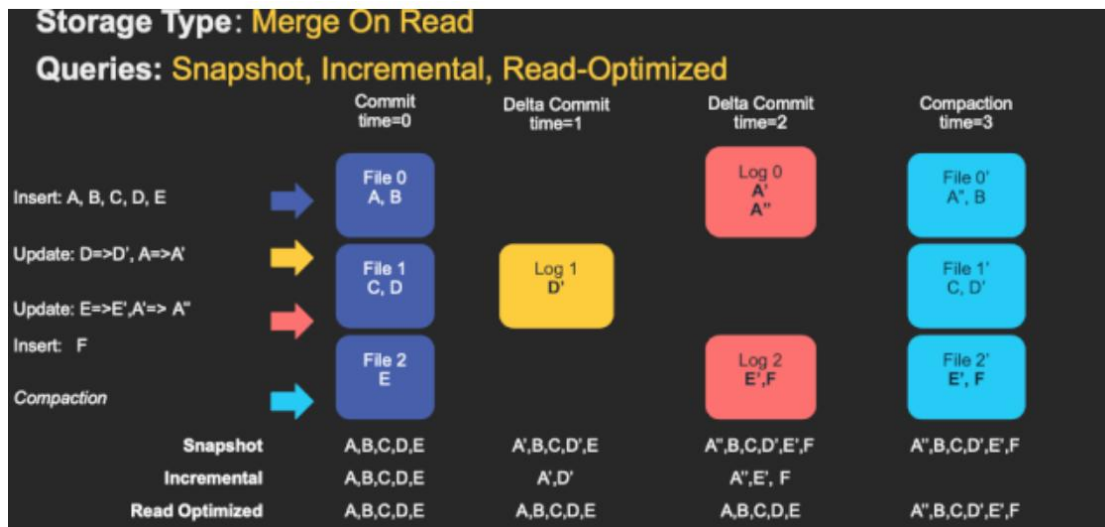
COW 表写的时候数据直接写入 basefile, (parquet) 不写 log 文件。所以 COW 表的文件片只包含 basefile(一个 parquet 文件构成一个文件片)。这种的存储方式的 Spark DAG 相对简单。关键目标是使用 **partitioner** 将 **tagged Hudi** 记录 RDD (所谓的 **tagged** 是指已经通过索引查询, 标记每条输入记录在表中的位置) 分成一些列的 **updates** 和 **inserts**。为了维护文件大小, 我们先对输入进行采样, 获得一个工作负载 **profile**, 这个 **profile** 记录了输入记录的 **insert** 和 **update**、以及在分区中的分布等信息。把数据重新打包, 这样: 1) 对于 **updates**, 该文件 ID 的最新版本都将被重写一次, 并对所有已更改的记录使用新值 2) 对于 **inserts**. 记录首先打包到每个分区路径中的最小文件中, 直到达到配置的最大大小。之后的所有剩余记录将再次打包到新的文件组, 新的文件组也会满足最大文件大小要求。



1.10.2. Merge On Read 表

MOR 表写数据时，记录首先会被快速的写进日志文件，稍后会使用时间轴上的压缩操作将其与基础文件合并。根据查询是读取日志中的合并快照流还是变更流，还是仅读取未合并的基础文件，MOR 表支持多种查询类型。在高层次上，MOR writer 在读取数据时会经历与 COW writer 相同的阶段。这些更新将追加到最新文件篇的最新日志文件中，而不会合并。对于 insert, Hudi 支持两种模式：1) 插入到日志文件：有可索引日志文件的表会执行此操作（HBase 索引）；2) 插入 parquet 文件：没有索引文件的表（例如布隆索引）与写时复制（COW）一样，对已标记位置的输入记录进行分区，以便将所有发往相同文件 id 的 upsert 分到一组。这批 upsert 会作为一个或多个日志块写入日志文件。Hudi 允许客户端控制日志文件大小。对于写时复制（COW）和读时合并（MOR）writer 来说，Hudi 的 WriteClient 是相同的。几轮数据的写入将会累积一个或多个日志文件。这些日志文件与基本的 parquet 文件（如有）一起构成一个文件片，而这个文件片代表该文件的一个完整版本。这种表是用途最广、最高级的表。为写（可以指定不同的压缩策略，吸收突发写流量）和查询（例如权衡数据的新鲜度和查询性能）提供了很大的灵活性。同时它包含一个学习曲线，以便在操作上掌控他。





1.10.3. 写时复制 (COW) 与读时合并 (MOR) 存储类型有什么区别?

写入时复制- 这种存储类型使客户端能够以柱状文件格式 (目前为镶木地板) 摄取数据。使用 COW 存储类型写入 Hudi 数据集的任何新数据都将写入新的 Parquet 文件。更新现有的一组行将导致重写整个 Parquet 文件, 这些文件共同包含正在更新的受影响行。因此, 对此类数据集的所有写入都受到 parquet 写入性能的限制, parquet 文件越大, 摄取数据所需的时间就越长。

读取时合并- 这种存储类型使客户端能够将数据快速摄取到基于行的数据格式, 例如 avro。使用 MOR 表类型写入 Hudi 数据集的任何新数据都将写入新的日志/增量文件, 这些文件在内部将数据存储为 avro 编码字节。压缩过程 (配置为内联或异步) 会将日志文件格式转换为柱状文件格式 (parquet)。两个不同的 InputFormats 公开了此数据的 2 个不同视图, Read Optimized 视图公开了柱状镶木地板读取性能, 而实时视图分别公开了柱状和/或日志读取性能。更新现有的一组行将导致 a) 从先前压缩生成的现有基本镶木地板文件的伴随日志/增量文件或 b) 写入日志/增量文件的更新, 以防它没有发生压缩。因此, 所有对此类数据集的写入都受到 avro/log 文件写入性能的限制, 比 parquet 快得多。尽管如此, 读取日志/增量文件与柱状 (镶木地板) 文件相比, 需要支付更高的成本

1.11. 写原理

1.11.1. 写操作

了解 Hudi 数据源或者 deltastreamer 工具提供的 3 种不同写操作以及如何最好的利用他们可能会有所帮助。这些操作可以在对数据集发出的每个 commit/delta commit 中进行选择/更改。

1) upsert 操作: 这是默认操作, 在该操作中, 首先通过查询索引将数据记录标记为插入或更新, 然后再运行试探法确定如何最好地将他们打包到存储, 以对文件大小进行优化, 最终将记录写入。对于诸如数据库更改捕获之类的用例, 建议在输入几乎肯定包含更新的情况下

使用此操作。2) insert 操作: 与 upsert 相比, insert 操作也会运行试探法确定打包方式, 优化文件大小, 但会完全跳过索引查询。因此对于诸如日志重复数据删除(结合下面提到的过滤重复项选项)的用例而言, 它比 upsert 的速度快得多。这也适用于数据集可以容忍重复项, 但只需要 Hudi 具有事务性写/增量拉取/存储管理功能的用例。3) bulk insert 操作: upsert 和 insert 操作都会将输入记录保留在内存中, 以加快存储启发式计算速度, 因此对于最初加载/引导 Hudi 数据集的用例而言可能会很麻烦。Bulk insert 提供了与 insert 相同的语义, 同时实现了基于排序的数据写入算法, 该算法可以很好的扩展数百 TB 的初始负载。但是这只是在调整文件大小方面进行的最大努力, 而不是像 insert/update 那样保证文件大小。

1. 11. 2. 压缩

压缩是一个 instant 操作, 它将一组文件片作为输入, 将每个文件切片中的所有日志文件与其 basefile 文件(parquet 文件)合并, 以生成新的压缩文件片, 并写为时间轴上的一个 commit。压缩仅适用于读时合并(MOR)表类型, 并且由压缩策略(默认选择具有最大未压缩日志的文件片)决定选择要进行压缩的文件片。这个压缩策略会在每个写操作之后评估。从高层次上讲, 压缩有两种方式: 1) 同步压缩: 这里的压缩由写程序进程本身在每次写入之后同步执行的, 即直到压缩完成后才能开始下一个写操作。就操作而言, 这个是最简单的, 因为无需安排单独的压缩过程, 但保证的数据新鲜度最低。不过, 如果可以在每次写操作中压缩最新的表分区, 同时又能延迟迟到/较旧分区的压缩, 这种方式仍然非常有用。2) 异步压缩: 使用这种方式, 压缩过程可以与表的写操作同时异步运行。这样具有明显的好处, 即压缩不会阻塞下一批数据写入, 从而产生近实时的数据新鲜度。Hudi DeltaStreamer 之类的工具支持边界的连续模式, 其中的压缩和写入操作就是以这种方式在单个 Spark 运行时集群中进行的。

1. 11. 3. 清理

清理是一项基本的即时操作, 其执行的目的是删除旧的文件片, 并限制表占用的存储空间。清理会在每次写操作之后自动执行, 并利用时间轴服务器上缓存的时间轴元数据来避免扫描整个表来评估清理时机。Hudi 支持两种清理方式: 1) 按 commits / deltacommits 清理: 这是增量查询中最常见且必须使用的模式。以这种方式, Cleaner 会保留最近 N 次 commit/delta commit 提交中写入的所有文件切片, 从而有效提供在任何即时范围内进行增量查询的能力。尽管这对于增量查询很有帮助, 但由于保留了配置范围内所有版本的文件片, 因此, 在某些高写入负载的场景下可能需要更大的存储空间。2) 按保留的文件片清理: 这是一种更为简单的清理方式, 这里我们仅保存每个文件组中的最后 N 个文件片。诸如 Apache Hive 之类的某些查询引擎会处理非常大的查询, 这些查询可能需要几个小时才能完成, 在这种情况下, 将 N 设置为足够大以至于不会删除查询仍然可以访问的文件片是很有用的。此外, 清理操作会保证每个文件组下面会一直只有一个文件片(最新的一片)。

1. 11. 4. DFS 访问优化

Hudi 还对表中存储的数据执行了几种秘钥存储管理功能。在 DFS 上存储数据的关键是管理文件大小和计数以及回收存储空间。例如, HDFS 在处理小文件问题上臭名昭著--在

NameNode 上施加内存/RPC 压力, 可能破坏整个集群的稳定性。通常, 查询引擎可在适当大小的列文件上提供更好的性能, 因为它们可以有效地摊销获取列统计信息等的成本。即使在某些云数据存储上, 列出包含大量小文件的目录也会产生成本。下面是一些 Hudi 高效写, 管理数据存储的方法: 1) 小文件处理特性会剖析输入的工作负载, 并将内容分配到现有的文件组, 而不是创建新文件组 (这会导致生成小文件)。2) 在 `writer` 中使用一个时间轴缓存, 这样只要 Spark 集群不每次都重启, 后续的写操作就不需要列出 DFS 目录来获取指定分区路径下的文件片列表。3) 用户还可以调整基本文件和日志文件大小之间的比值系数以及期望的压缩率, 以便将足够数量的 `insert` 分到统一文件组, 从而生成大小合适的基本文件。4) 智能调整 `bulk insert` 并行度, 可以再次调整大小合适的初始文件组。实际上, 正确执行此操作非常关键, 因为文件组一旦创建就不能被删除, 而只能如前面所述对其进行扩展。

1.12. 读取数据原理

1.12.1. 快照查询

可查看给定 `delta commit` 或者 `commit` 即时操作后表的最新快照。在读时合并 (MOR) 表的情况下, 它通过即时合并最新文件片的基本文件和增量文件来提供近实时表 (几分钟)。对于写时复制 (COW), 它可以替代现有的 `parquet` 表 (或相同基本文件类型的表), 同时提供 `upsert/delete` 和其他写入方面的功能。

1.12.2. 增量查询

可查看自给定 `commit/delta commit` 即时操作以来新写入的数据。有效的提供变更流来启用增量数据管道。

1.12.3. 读优化查询

可查看给定的 `commit/compact` 即时操作的表的最新快照。仅将最新文件片的基本/列文件暴露给查询, 并保证与非 Hudi 表相同的列查询性能。指标读优化查询快照查询数据延迟高低查询延迟低高

第三篇 实战案例

1.13. hudi 数据同步到 hive

注意案例中使用的环境:

spark 3.0.0

hadoop 2.7.3

hive 2.3.1

建议将 hudi 的 jar 包复制到 hive 的 lib 目录下

```
hudi-hadoop-mr-bundle-0.9.0.jar  
hudi-hive-sync-bundle-0.9.0.jar
```

重启 hive 的元数据服务

重启 hiveserver2 远程连接服务

1. 13. 1. 添加依赖

```
<properties>  
    <maven.compiler.source>8</maven.compiler.source>  
    <maven.compiler.target>8</maven.compiler.target>  
</properties>  
<dependencies>  
    <dependency>  
        <groupId>joda-time</groupId>  
        <artifactId>joda-time</artifactId>  
        <version>2.9.9</version>  
    </dependency>  
    <dependency>  
        <groupId>org.apache.hadoop</groupId>  
        <artifactId>hadoop-client</artifactId>  
        <version>2.7.3</version>  
    </dependency>  
  
    <!-- https://mvnrepository.com/artifact/org.apache.spark/spark-core -->  
    <dependency>  
        <groupId>org.apache.spark</groupId>  
        <artifactId>spark-core_2.12</artifactId>  
        <version>3.0.0</version>  
    </dependency>  
  
    <dependency>  
        <groupId>org.apache.spark</groupId>  
        <artifactId>spark-sql_2.12</artifactId>  
        <version>3.0.0</version>  
    </dependency>  
    <dependency>  
        <groupId>org.apache.spark</groupId>  
        <artifactId>spark-hive_2.12</artifactId>  
        <version>3.0.0</version>  
    </dependency>
```

```
<dependency>
  <groupId>org.apache.spark</groupId>
  <artifactId>spark-avro_2.12</artifactId>
  <version>3.0.0</version>
</dependency>
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <version>5.1.48</version>
</dependency>

<dependency>
  <groupId>com.doitedu</groupId>
  <artifactId>hudi_spark</artifactId>
  <version>12</version>
  <scope>system</scope>
  <systemPath>${project.basedir}/lib/hudi-spark3-bundle_2.12-0.9.0.jar</systemPath>
>

</dependency>
<!-- https://mvnrepository.com/artifact/org.apache.hive/hive-jdbc -->
<dependency>
  <groupId>org.apache.hive</groupId>
  <artifactId>hive-jdbc</artifactId>
  <version>2.3.1</version>
</dependency>
</dependencies>
<!-- maven 插件下载国内镜像库 -->
<pluginRepositories>
  <pluginRepository>
    <id>ali-plugin</id>
    <url>http://maven.aliyun.com/nexus/content/groups/public/</url>
    <snapshots>
      <enabled>false</enabled>
      <updatePolicy>never</updatePolicy>
    </snapshots>
    <releases>
      <enabled>true</enabled>
      <updatePolicy>never</updatePolicy>
    </releases>
  </pluginRepository>
</pluginRepositories>
<build>
  <plugins>
    <!-- 指定编译 java 的插件 -->
```



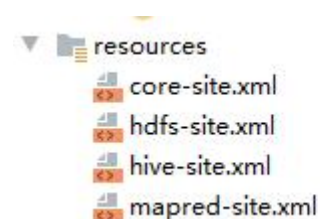
```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <version>3.5.1</version>
  <configuration>
    <source>1.8</source>
    <target>1.8</target>
  </configuration>
</plugin>

<plugin>
  <artifactId>maven-assembly-plugin</artifactId>
  <configuration>
    <descriptorRefs>
      <descriptorRef>jar-with-dependencies</descriptorRef>
    </descriptorRefs>
  </configuration>
  <executions>
    <execution>
      <id>make-assembly</id>
      <phase>package</phase>
      <goals>
        <goal>single</goal>
      </goals>
    </execution>
  </executions>
</plugin>
</plugins>
</build>

</project>
```

1.13.2. hive 环境测试

将配置文件同步到项目资源路径下



```
package com.doitedu.demo.hive
```

```
import org.apache.hudi.DataSourceWriteOptions
import org.apache.spark.SparkConf
import org.apache.spark.sql.types.{DataTypes, StructType}
import org.apache.spark.sql.{DataFrame, SparkSession}

/**
 * @Date 2021/11/1
 * @Created by HANGGE
 * @Description
 *      测试 spark 操作 hive 的环境
 */
object C00_SycHive {
  def main(args: Array[String]): Unit = {
    System.setProperty("HADOOP_USER_NAME", "root");
    val session = SparkSession.builder()
      .master("local[*]")
      .appName("hive_demo")
      .enableHiveSupport() // 开启对 hive 的支持
      .getOrCreate()

    /**
     * 注意
     * 1 直接编写 sql 语句即可操作 hive 自动的读取 resources 下的三个配置文件
     *   1) hive 的元数据记录在 lx01 机器的 mysql 中 lx01 的 mysql 正常启动
     *   2) hive 的数据存储在 HDFS 中的 hdfs 正常启动
     * 2 但是不要写 ;
     * 3 select * from tb_name 直接操作的是 default 数据库中的表
     * 4 操作自己的数据库中的表 数据库.表名
     */
    session.sql("show databases").show() //
    session.stop()

  }
}
```

```
7067 [main] INFO org.apache.spark.sql.catalyst.expressions.codegen.CodeGenerator - Code generated in 6.4967 ms
+-----+
|namespace|
+-----+
| default|
+-----+
```

出现上面的结果 ,说明 spark 操作 hive 是 OK 的!

1.13.3. 无分区表

```
package com.doitedu.demo.hive

import org.apache.hudi.DataSourceWriteOptions
import org.apache.hudi.config.HoodieIndexConfig
import org.apache.hudi.hive.MultiPartKeysValueExtractor
import org.apache.hudi.index.HoodieIndex
import org.apache.spark.SparkConf
import org.apache.spark.sql.types.{DataTypes, StructType}
import org.apache.spark.sql.{DataFrame, SaveMode, SparkSession}

/**
 * @Date 2021/11/1
 * @Created by HANGGE
 * @Description
 *      处理数据将数据写入 hudi 同步到 hive 中
 *      --指定字符串为分区字段
 */
object C02_SychHiveNoPartition {
  def main(args: Array[String]): Unit = {
    System.setProperty("HADOOP_USER_NAME", "root");
    val sparkConf = new
    SparkConf().setAppName("testhive").setMaster("local[*]")
      .set("spark.serializer",
"org.apache.spark.serializer.KryoSerializer")
    val sparkSession =
    SparkSession.builder().config(sparkConf).enableHiveSupport().getOrCreate()
    insertData(sparkSession);
  }
  /**
   * 插入数据 指定多个分区字段
   * @param sparkSession
   */
  def insertData(sparkSession: SparkSession) = {
    import org.apache.hudi.QuickstartUtils._
    import org.apache.spark.sql.functions._

    val commitTime = System.currentTimeMillis().toString //生成提交时间
    val schema = new StructType()
      .add("id", DataTypes.StringType)
      .add("name", DataTypes.StringType)
      .add("age", DataTypes.IntegerType)
```

```
.add("city", DataTypes.StringType)
.add("score", DataTypes.DoubleType)
.add("gender", DataTypes.StringType)
// 加载数据同时添加一个 时间字段
val df: DataFrame =
sparkSession.read.schema(schema).csv("/data/user.txt")
    .withColumn("ts", lit(commitTime))
df.show

// 将数据写入到 HUDI 同时同步到 hive 中
df.write.format("org.apache.hudi")
    .options(getQuickstartWriteConfigs())
    .option(DataSourceWriteOptions.TABLE_TYPE.key(), "COPY_ON_WRITE") //
指定表的模式为行
    .option(DataSourceWriteOptions.RECORDKEY_FIELD.key(), "id") //设置主
键
    .option(DataSourceWriteOptions.PRECOMBINE_FIELD.key(), "ts") // 数据
更新时间
    // 没有分区指定
    .option(DataSourceWriteOptions.KEYGENERATOR_CLASS_NAME.key(), ,
"org.apache.hudi.keygen.NonpartitionedKeyGenerator")
    .option("hoodie.table.name", "tb_user_no_par") //hudi 表名
    .option(DataSourceWriteOptions.HIVE_URL.key(),
"jdbc:hive2://doit01:10000") // 连接 hive 地址
    .option(DataSourceWriteOptions.HIVE_USER.key(), "root")
    .option(DataSourceWriteOptions.HIVE_PASS.key(), "")
    .option(DataSourceWriteOptions.HIVE_DATABASE.key(), "default") //设置
hudi 与 hive 同步的数据库
    .option(DataSourceWriteOptions.HIVE_TABLE.key(), "tb_user_no_par") //
设置 hudi 与 hive 同步的表名
    .option(DataSourceWriteOptions.HIVE_SYNC_ENABLED.key(), "true") //设
置数据集注册并同步到 hive
    .option(HoodieIndexConfig.INDEX_TYPE.key(),
HoodieIndex.IndexType.GLOBAL_BLOOM.name())
    //设置索引类型目前有 HBASE, INMEMORY, BLOOM, GLOBAL_BLOOM 四种索引 为了保
证分区变更后能找到必须设置全局 GLOBAL_BLOOM
    .option("hoodie.insert.shuffle.parallelism", "12")
    .option("hoodie.upsert.shuffle.parallelism", "12")
    //指定分区提取类
    .option(DataSourceWriteOptions.HIVE_PARTITION_EXTRACTOR_CLASS.key(),
"org.apache.hudi.hive.NonPartitionedExtractor")
    .mode(SaveMode.Append)
    .save("/hudi/tb_user_no_par");
}
```

```
}
```

1.13.4. 指定字符串字段为分区

```
package com.doitedu.demo.hive

import org.apache.hudi.DataSourceWriteOptions
import org.apache.hudi.config.HoodieIndexConfig
import org.apache.spark.SparkConf
import org.apache.spark.sql.{DataFrame, SaveMode, SparkSession}
import org.apache.spark.sql.types.{DataTypes, StructType}
import org.apache.hudi.config.HoodieWriteConfig.TBL_NAME
import org.apache.hudi.index.HoodieIndex
/**
 * @Date 2021/11/1
 * @Created by HANGGE
 * @Description
 *      处理数据将数据写入 hudi 同步到 hive 中
 *      --指定字符串为分区字段
 */
object C01_SycHive {
  def main(args: Array[String]): Unit = {
    System.setProperty("HADOOP_USER_NAME", "root");
    val sparkConf = new SparkConf().setAppName("testhive").setMaster("local[*]")
      .set("spark.serializer", "org.apache.spark.serializer.KryoSerializer")
    val sparkSession =
      SparkSession.builder().config(sparkConf).enableHiveSupport().getOrCreate()
    insertData(sparkSession);
  }
  /**
   * 插入数据 指定多个分区字段
   * @param sparkSession
   */
  def insertData(sparkSession: SparkSession) = {
    import org.apache.hudi.QuickstartUtils._
    import org.apache.hudi.config.HoodieWriteConfig._
    import org.apache.spark.sql.SaveMode._
    import org.apache.spark.sql.functions._

    val commitTime = System.currentTimeMillis().toString //生成提交时间
    val schema = new StructType()
      .add("id", DataTypes.StringType)
```

```
.add("name", DataTypes.StringType)
.add("age", DataTypes.IntegerType)
.add("city", DataTypes.StringType)
.add("score", DataTypes.DoubleType)
.add("gender", DataTypes.StringType)
// 加载数据同时添加一个 时间字段
val df: DataFrame = sparkSession.read.schema(schema).csv("/data/user.txt")
    .withColumn("ts", lit(commitTime))
df.show

// 将数据写入到 HUDI 同时同步到 hive 中
df.write.format("org.apache.hudi")
    .options(getQuickstartWriteConfigs())
    .option(DataSourceWriteOptions.TABLE_TYPE.key(), "COPY_ON_WRITE") // 指定表的
模式乐行
    .option(DataSourceWriteOptions.RECORDKEY_FIELD.key(), "id") //设置主键
    .option(DataSourceWriteOptions.PRECOMBINE_FIELD.key(), "ts") // 数据更新时间
    .option(DataSourceWriteOptions.PARTITIONPATH_FIELD.key(), "city") //设置 hudi
中的分区字段
    .option("hoodie.table.name", "tb_user") //hudi 表名
    .option(DataSourceWriteOptions.HIVE_URL.key(), "jdbc:hive2://doit01:10000")
// 连接 hive 地址
    .option(DataSourceWriteOptions.HIVE_USER.key(), "root")
    .option(DataSourceWriteOptions.HIVE_PASS.key(), "")
    .option(DataSourceWriteOptions.HIVE_DATABASE.key(), "default") //设置 hudi 与
hive 同步的数据库
    .option(DataSourceWriteOptions.HIVE_TABLE.key(), "tb_user") //设置 hudi 与 hive
同步的表名
    .option(DataSourceWriteOptions.HIVE_SYNC_ENABLED.key(), "true") //设置数据集注
册并同步到 hive
    .option(HoodieIndexConfig.INDEX_TYPE.key(),
HoodieIndex.IndexType.GLOBAL_BLOOM.name())
    .option("hoodie.datasource.hive_sync.partition_fields", "city") // 指定 hive
中的分区字段 ,在建表的时候会创建这个分区字段
    //设置索引类型目前有 HBASE,INMEMORY,BLOOM,GLOBAL_BLOOM 四种索引 为了保证分区变更
后能找到必须设置全局 GLOBAL_BLOOM
    .option("hoodie.insert.shuffle.parallelism", "12")
    .option("hoodie.upsert.shuffle.parallelism", "12")
//指定分区提取类
    .option(DataSourceWriteOptions.HIVE_PARTITION_EXTRACTOR_CLASS.key(),
"org.apache.hudi.hive.MultiPartKeyValueExtractor")
    .mode(SaveMode.Append)
    .save("/hudi/tb_user");
```

```
}  
}
```

测试查看数据

```
show tables ;  
+-----+  
| tab_name |  
+-----+  
| tb_user  |  
+-----+  
select * from tb_user ;  
+-----+-----+-----+-----+-----+-----+  
| id | name | age | gender | score | city |  
+-----+-----+-----+-----+-----+-----+  
| 1  | 黄渤 | 29  | M      | 88.0  | BJ   |  
| 2  | 韩雪 | 44  | F      | 77.0  | BJ   |  
| 3  | 杨紫 | 31  | F      | 89.0  | SH   |  
+-----+-----+-----+-----+-----+-----+  
show partitions tb_user ;  
+-----+  
| partition |  
+-----+  
| city=BJ   |  
| city=SH   |  
+-----+
```

1.13.5. 多级分区

```
package com.doitedu.demo.hive  
  
import org.apache.hudi.DataSourceWriteOptions  
import org.apache.hudi.config.HoodieIndexConfig  
import org.apache.hudi.index.HoodieIndex  
import org.apache.spark.SparkConf  
import org.apache.spark.sql.types.{DataTypes, StructType}  
import org.apache.spark.sql.{DataFrame, SaveMode, SparkSession}  
  
/**  
 * @Date 2021/11/1  
 * @Created by HANGGE  
 * @Description  
 *          处理数据将数据写入 hudi 同步到 hive 中  
 */
```



```
*      --指定字符串为分区字段
*/

object C03_SycHiveMultPartition {
  def main(args: Array[String]): Unit = {
    System.setProperty("HADOOP_USER_NAME", "root");
    val sparkConf = new SparkConf().setAppName("testhive").setMaster("local[*]")
      .set("spark.serializer", "org.apache.spark.serializer.KryoSerializer")
    val sparkSession = SparkSession.builder().config(sparkConf).enableHiveSupport().getOrCreate()
    insertData(sparkSession);
  }
  /**
   * 插入数据 指定多个分区字段
   * @param sparkSession
   */
  def insertData(sparkSession: SparkSession) = {
    import org.apache.hudi.QuickstartUtils._
    import org.apache.spark.sql.functions._

    val commitTime = System.currentTimeMillis().toString //生成提交时间
    val schema = new StructType()
      .add("id", DataTypes.StringType)
      .add("name", DataTypes.StringType)
      .add("age", DataTypes.IntegerType)
      .add("city", DataTypes.StringType)
      .add("score", DataTypes.DoubleType)
      .add("gender", DataTypes.StringType)
    // 加载数据同时添加一个 时间字段
    val df: DataFrame = sparkSession.read.schema(schema).csv("/data/user.txt")
      .withColumn("ts", lit(commitTime))
    df.show

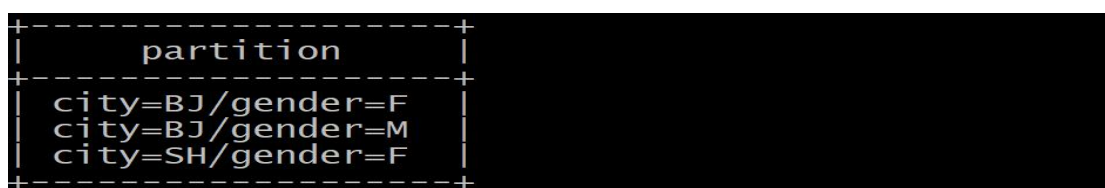
    // 将数据写入到 HUDI 同时同步到 hive 中
    df.write.format("org.apache.hudi")
      .options(getQuickstartWriteConfigs())
      .option(DataSourceWriteOptions.TABLE_TYPE.key(), "COPY_ON_WRITE") // 指定表的模式为行
      .option(DataSourceWriteOptions.RECORDKEY_FIELD.key(), "id") //设置主键
      .option(DataSourceWriteOptions.PRECOMBINE_FIELD.key(), "ts") // 数据更新时间
    // 写数据的时候执行分区
    .option(DataSourceWriteOptions.PARTITIONPATH_FIELD.key(), "city,gender")
    .option(DataSourceWriteOptions.KEYGENERATOR_CLASS_NAME.key(),
      "org.apache.hudi.keygen.ComplexKeyGenerator")
    .option("hoodie.table.name", "tb_user_muti_par") //hudi 表名
    .option(DataSourceWriteOptions.HIVE_URL.key(), "jdbc:hive2://doit01:10000") // 连接 hive 地址
    .option(DataSourceWriteOptions.HIVE_USER.key(), "root")
```

```
.option(DataSourceWriteOptions.HIVE_PASS.key(), "")
.option(DataSourceWriteOptions.HIVE_DATABASE.key(), "default") //设置 hudi 与 hive 同步的数据库
.option(DataSourceWriteOptions.HIVE_TABLE.key(), "tb_user_muti_par") //设置 hudi 与 hive 同步的
表名

// 指定同步 hive 的分区
.option("hoodie.datasource.hive_sync.partition_fields", "city,gender")
.option(DataSourceWriteOptions.HIVE_SYNC_ENABLED.key(), "true") //设置数据集注册并同步到 hive
.option(HoodieIndexConfig.INDEX_TYPE.key(), HoodieIndex.IndexType.GLOBAL_BLOOM.name())
//设置索引类型目前有 HBASE, INMEMORY, BLOOM, GLOBAL_BLOOM 四种索引 为了保证分区变更后能找到必须设置全
局 GLOBAL_BLOOM

.option("hoodie.insert.shuffle.parallelism", "12")
.option("hoodie.upsert.shuffle.parallelism", "12")
//指定分区提取类
.option(DataSourceWriteOptions.HIVE_PARTITION_EXTRACTOR_CLASS.key(),
"org.apache.hudi.hive.MultiPartKeysValueExtractor")
.mode(SaveMode.Append)
.save("/hudi/tb_user_muti_par");

}
}
```



/hudi/tb_user_muti_par/BJ							Go!
Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
drwxr-xr-x	root	supergroup	0 B	2021/11/3 下午9:44:27	0	0 B	F
drwxr-xr-x	root	supergroup	0 B	2021/11/3 下午9:44:28	0	0 B	M

1.13.6. 时间格式分区

① 数据

1,2021-11-01,88

2,2021-11-02,99

3,2021-11-01,100

4,2021-11-02,89

5,2021-10-30,33

② 自定义分区数据抽取器

hudi 内存支持的分区数据抽取器有下列几种

```
public interface PartitionValueExtractor extends Serializable {
    List<String> extractPartitionValuesInPath(String partitionPath);
}

Choose Implementation of PartitionValueExtractor (6 found)
- HiveStylePartitionValueExtractor (org.apache.hudi.hive) Maven: com.doitedu:hudi_spark:12 (hudi-spark)
- MultiPartKeysValueExtractor (org.apache.hudi.hive) Maven: com.doitedu:hudi_spark:12 (hudi-spark)
- MyExtractor (com.doitedu.demo.hive)
- NonPartitionedExtractor (org.apache.hudi.hive) Maven: com.doitedu:hudi_spark:12 (hudi-spark)
- SlashEncodedDayPartitionValueExtractor (org.apache.hudi.hive) Maven: com.doitedu:hudi_spark:12 (hudi-spark)
- SlashEncodedHourPartitionValueExtractor (org.apache.hudi.hive) Maven: com.doitedu:hudi_spark:12 (hudi-spark)

/**
 * @Date 2021/11/3
 * @Created by HANGGE
 * @Description TODO
 */

public class MyExtractor implements PartitionValueExtractor {

    private static final long serialVersionUID = 1L;

    private transient DateTimeFormatter dtfOut = DateTimeFormatter.ofPattern("yyyy-MM-dd");

    public MyExtractor() {

    }

    private DateTimeFormatter getDtfOut() {

        if (this.dtfOut == null) {

            this.dtfOut = DateTimeFormatter.ofPattern("yyyy-MM-dd");

        }

        return this.dtfOut;
    }

    public List<String> extractPartitionValuesInPath(String partitionPath) {

        String[] splits = partitionPath.split("-");

        if (splits.length != 3) {

            throw new IllegalArgumentException("Partition path " + partitionPath + " is not in the form yyyy/mm/dd");

        } else {

            int year = Integer.parseInt(splits[0].contains("=") ? splits[0].split("=")[1] : splits[0]);

            int mm = Integer.parseInt(splits[1].contains("=") ? splits[1].split("=")[1] : splits[1]);

            int dd = Integer.parseInt(splits[2].contains("=") ? splits[2].split("=")[1] : splits[2]);

            ZonedDateTime dateTime = ZonedDateTime.of(LocalDate.of(year, mm, dd), Time.of(0, 0), ZoneId.systemDefault());

            return Collections.singletonList(dateTime.format(this.getDtfOut()));

        }

    }

}
```

③ 同步数据

```
package com.doitedu.demo.hive

import org.apache.hudi.DataSourceWriteOptions
import org.apache.hudi.config.HoodieIndexConfig
import org.apache.hudi.hive.SlashEncodedDayPartitionValueExtractor
import org.apache.hudi.index.HoodieIndex
import org.apache.spark.SparkConf
import org.apache.spark.sql.types.{DataTypes, StructType}
import org.apache.spark.sql.{DataFrame, SaveMode, SparkSession}

/**
 * @Date 2021/11/1
 * @Created by HANGGE
 * @Description
 *      处理数据将数据写入 hudi 同步到 hive 中
 *      --指定字符串为分区字段
 */
object C04_SycHiveDatePartition {
  def main(args: Array[String]): Unit = {
    System.setProperty("HADOOP_USER_NAME", "root");

    val sparkConf = new SparkConf().setAppName("testhive").setMaster("local[*]")
      .set("spark.serializer", "org.apache.spark.serializer.KryoSerializer")

    val sparkSession = SparkSession.builder().config(sparkConf).enableHiveSupport().getOrCreate()

    insertData(sparkSession);
  }

  /**
   * 插入数据 指定多个分区字段
   * @param sparkSession
   */
  def insertData(sparkSession: SparkSession) = {
    import org.apache.hudi.QuickstartUtils._
    import org.apache.spark.sql.functions._

    val commitTime = System.currentTimeMillis().toString //生成提交时间
    val schema = new StructType()
      .add("id", DataTypes.StringType)
      .add("cdate", DataTypes.StringType)
      .add("money", DataTypes.DoubleType)
    // 加载数据同时添加一个 时间字段
    val df: DataFrame = sparkSession.read.schema(schema).csv("/data/orders.txt")
      .withColumn("ts", lit(commitTime))

    df.show

    // 将数据写入到 HUDI 同时同步到 hive 中
  }
}
```

```
df.write.format("org.apache.hudi")

.option(getQuickstartWriteConfigs())

.option(DataSourceWriteOptions.TABLE_TYPE.key(), "COPY_ON_WRITE") // 指定表的模式为行

.option(DataSourceWriteOptions.RECORDKEY_FIELD.key(), "id") //设置主键

.option(DataSourceWriteOptions.PRECOMBINE_FIELD.key(), "ts") // 数据更新时间

// 写数据的时候执行分区

.option(DataSourceWriteOptions.PARTITIONPATH_FIELD.key(), "cdate")

.option(DataSourceWriteOptions.KEYGENERATOR_CLASS_NAME.key(), "org.apache.hudi.keygen.ComplexKeyGenerator")

.option("hoodie.table.name", "tb_user_date_par") //hudi 表名

.option(DataSourceWriteOptions.HIVE_URL.key(), "jdbc:hive2://doit01:10000") // 连接 hive 地址

.option(DataSourceWriteOptions.HIVE_USER.key(), "root")

.option(DataSourceWriteOptions.HIVE_PASS.key(), "")

.option(DataSourceWriteOptions.HIVE_DATABASE.key(), "default") //设置 hudi 与 hive 同步的数据库

.option(DataSourceWriteOptions.HIVE_TABLE.key(), "tb_user_date_par") //设置 hudi 与 hive 同步的表名

// 指定同步 hive 的分区

.option("hoodie.datasource.hive_sync.partition_fields", "cdate")

.option(DataSourceWriteOptions.HIVE_SYNC_ENABLED.key(), "true") //设置数据集注册并同步到 hive

.option(HoodieIndexConfig.INDEX_TYPE.key(), HoodieIndex.IndexType.GLOBAL_BLOOM.name())

//设置索引类型目前有 HBASE, INMEMORY, BLOOM, GLOBAL_BLOOM 四种索引 为了保证分区变更后能找到必须设置全局 GLOBAL_BLOOM

.option("hoodie.insert.shuffle.parallelism", "12")

.option("hoodie.upsert.shuffle.parallelism", "12")

//指定分区提取类

.option("hoodie.datasource.hive_sync.partition_extractor_class", "com.doitedu.demo.hive.MyExtractor")

.mode(SaveMode.Append)

.save("/hudi/tb_user_date_par");

}

}
```

/hudi/tb_user_date_par Go!

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
drwxr-xr-x	root	supergroup	0 B	2021/11/3 下午10:52:22	0	0 B	.hoodie
drwxr-xr-x	root	supergroup	0 B	2021/11/3 下午10:52:21	0	0 B	2021-10-30
drwxr-xr-x	root	supergroup	0 B	2021/11/3 下午10:52:21	0	0 B	2021-11-01
drwxr-xr-x	root	supergroup	0 B	2021/11/3 下午10:52:21	0	0 B	2021-11-02

1.13.7. 更新数据

```
package com.doitedu.demo.hive

import com.doitedu.demo.hive.C01_SycHive.insertData
import org.apache.hudi.DataSourceWriteOptions
import org.apache.hudi.QuickstartUtils.getQuickstartWriteConfigs
```

```
import org.apache.hudi.config.HoodieWriteConfig.TBL_NAME
import org.apache.spark.SparkConf
import org.apache.spark.sql.SaveMode.Append
import org.apache.spark.sql.types.{DataTypes, StructType}
import org.apache.spark.sql.{DataFrame, SaveMode, SparkSession}

/**
 * @Date 2021/10/31
 * @Created by HANGGE
 * @Description
 *      更新数据
 *      加载 DF 数据 将 DF 数据写入到 hudi 中
 *      当主键分区内主键相同的数据会被更新
 */
object C05_UpdateData {

  def main(args: Array[String]): Unit = {
    System.setProperty("HADOOP_USER_NAME", "root");
    val sparkConf = new SparkConf().setAppName("testhive").setMaster("local[*]")
      .set("spark.serializer", "org.apache.spark.serializer.KryoSerializer")
    val sparkSession = SparkSession.builder().config(sparkConf).enableHiveSupport().getOrCreate()
    insertData(sparkSession);
  }

  /**
   * 插入数据 指定多个分区字段
   * @param sparkSession
   */
  def insertData(sparkSession: SparkSession) = {
    import org.apache.hudi.QuickstartUtils._
    import org.apache.hudi.config.HoodieWriteConfig._
    import org.apache.spark.sql.SaveMode._
    import org.apache.spark.sql.functions._

    val commitTime = System.currentTimeMillis().toString //生成提交时间
    val schema = new StructType()
      .add("id", DataTypes.StringType)
      .add("name", DataTypes.StringType)
      .add("age", DataTypes.IntegerType)
      .add("city", DataTypes.StringType)
      .add("score", DataTypes.DoubleType)
      .add("gender", DataTypes.StringType)
    // 加载数据同时添加一个 时间字段
    val df: DataFrame = sparkSession.read.schema(schema).csv("/data/user3.txt")
      .withColumn("ts", lit(commitTime))
  }
}
```

```
df.show

df.write.format("org.apache.hudi")
  .options(getQuickstartWriteConfigs())
  .option(DataSourceWriteOptions.TABLE_TYPE.key(), "COPY_ON_WRITE")
  .option(DataSourceWriteOptions.RECORDKEY_FIELD.key(), "id") //设置主键
  .option(DataSourceWriteOptions.PRECOMBINE_FIELD.key(), "ts") // 数据更新时间
  .option(DataSourceWriteOptions.PARTITIONPATH_FIELD.key(), "city")
  .option("hoodie.table.name", "tb_user") //hudi 表名 //hudi 中的表名
  .mode(SaveMode.Append)
  .save("/hudi/tb_user")

sparkSession.close()
}
```

在 hive 中查询数据

```
select id , name , age , gender , score , city from tb_user ;
```

```
+-----+-----+-----+-----+-----+-----+
| id | name | age | gender | score | city |
+-----+-----+-----+-----+-----+-----+
| 1 | 黄渤 | 29 | M | 88.0 | BJ |
| 2 | 韩雪 | 44 | F | 77.0 | BJ |
| 3 | 杨紫 | 31 | F | 89.0 | SH |
+-----+-----+-----+-----+-----+-----+
```

```
hdfs dfs -cat /data/user3.txt
```

```
7,杨颖,44,BJ,77,F
```

```
8,杨过,31,SH,89,F
```

更新数据到 hudi 目录后, 查询 hive 中的数据 ,发现数据可以更新, 删除,

```
+-----+-----+-----+-----+-----+-----+
| id | name | age | gender | score | city |
+-----+-----+-----+-----+-----+-----+
| 1 | 黄渤 | 29 | M | 88.0 | BJ |
| 2 | 韩雪 | 44 | F | 77.0 | BJ |
| 7 | 杨颖 | 44 | F | 77.0 | BJ |
| 3 | 杨紫 | 31 | F | 89.0 | SH |
| 8 | 杨过 | 31 | F | 89.0 | SH |
+-----+-----+-----+-----+-----+-----+
```


1.13.8. 使用 shell 命令同步数据到 hive

1) 配置 hive 的环境

```
cp hudi-hive-sync-bundle-0.9.0.jar /opt/apps/hive-3.1.2/lib/
```

2) 重启 hive 服务

3) 修改脚本 jars

```
HIVE_JARS=${HIVE_HOME}/lib/*
```

```
./run_sync_tool.sh --help 查看脚本帮助命令
```

4 如果自己指定了分区字段 是字符串 指定 分区抽取器

```
--partition-value-extractor org.apache.hudi.hive.MultiPartKeyValueExtractor
```

```
./run_sync_tool.sh --jdbc-url jdbc:hive2://doit01:10000 --user root --pass " " --partitioned-by city
--base-path /hudi_dw/tb_user2 --database default --table tb_hudi_sync
--partition-value-extractor org.apache.hudi.hive.MultiPartKeyValueExtractor
```

--jdbc-url hive 远程连接地址

--user hive 的用户名

--pass 密码

--partitioned by 指定分区字段

--base-path hudi 表目录

--database hive 中的数据库, 默认会自动创建

--table hive 中的目标表 自动创建

--partition-value-extractor 自定义的抽取器

1.13.9. 使用 spark-SQL 更新数据

```
spark-sql --jars
/data/hudi-0.9.0/packaging/hudi-spark-bundle/target/hudi-spark3-bundle_2.12-0.9
.0.jar --packages
org.apache.hudi:hudi-spark3-bundle_2.12:0.9.0,org.apache.spark:spark-avro_2.12:3.
0.1 --conf 'spark.serializer=org.apache.spark.serializer.KryoSerializer'
--conf
'spark.sql.extensions=org.apache.spark.sql.hudi.HoodieSparkSessionExtension'

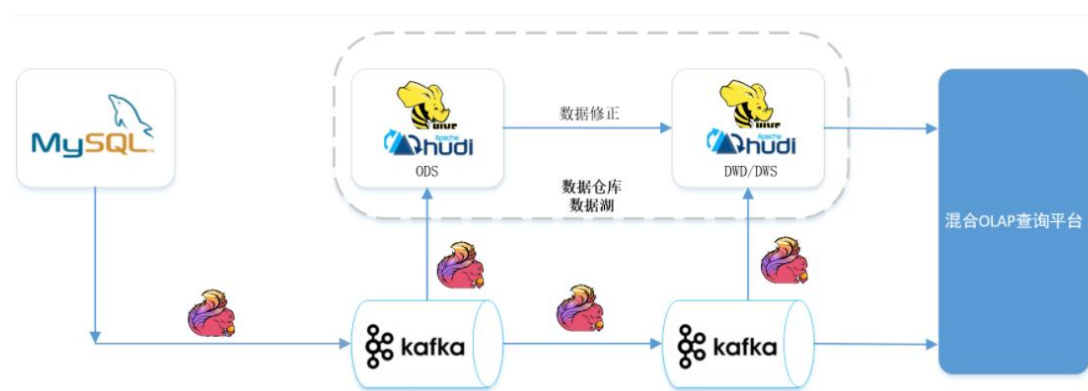
-- 建表 指定 hudi 中的路径
create table tb_user_no_par using hudi location '/hudi/tb_user_no_par' ;
-- 插入数据
insert into tb_user_no_par values (4,'NABI',27,'F',99,'SH' , '2021-10-11') ;

在 hive 中查询
+-----+-----+-----+-----+-----+
| id | name | age | gender | score | city |
+-----+-----+-----+-----+-----+

```

1	黄渤	29	M	88.0	BJ	
2	韩雪	44	F	77.0	BJ	
3	杨紫	31	F	89.0	SH	
4	NABI	27	SH	99.0	F	
+-----+-----+-----+-----+-----+-----+						

1.14. FlinkCDC+Hud 湖仓一体方案



1.14.1. 配置mysql的binlog

```
vi /etc/my.cnf
[mysqld]
server_id=1
log-bin=mysql-bin
binlog-format=ROW
-- 重启 mysql 的服务
service mysql restart ;
-- 登录 mysql 中 ,设置响应的参数
mysql> SET SESSION binlog_format = 'ROW';
mysql> SET GLOBAL binlog_format = 'ROW';
mysql> set session binlog_row_image=FULL;
mysql> set global binlog_row_image=FULL;
```

在 mysql 中创建表

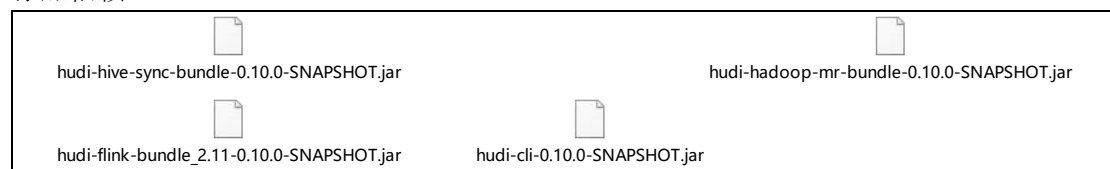
```
CREATE TABLE `tb_stu` (
  `uuid` varchar(20) DEFAULT NULL,
  `id` int(11) NOT NULL,
  `name` varchar(40) DEFAULT NULL,
  `description` varchar(40) DEFAULT NULL,
```

```
`ts` timestamp(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3)
) ENGINE=InnoDB DEFAULT CHARSET=latin1 |
```

-- 向表中插入数据

1.14.2. 搭建 flink 环境

添加依赖



```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.doitedu</groupId>
  <artifactId>hange-cdc</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>8</maven.compiler.source>
    <maven.compiler.target>8</maven.compiler.target>
    <flink.version>1.13.1</flink.version>
    <Hudi.version>0.10.0</Hudi.version>
    <hadoop.version>2.7.3</hadoop.version>
  </properties>

  <repositories>
    <repository>
      <id>apache.snapshots</id>
      <name>Apache Development Snapshot Repository</name>
      <url>https://repository.apache.org/content/repositories/snapshots/</url>
      <releases>
        <enabled>false</enabled>
      </releases>
      <snapshots>
        <enabled>true</enabled>
      </snapshots>
    </repository>
  </repositories>
</project>
```

```
</snapshots>
</repository>
</repositories>
<dependencies>

<dependency>
  <groupId>org.apache.hadoop</groupId>
  <artifactId>hadoop-client</artifactId>
  <version>${hadoop.version}</version>
</dependency>
<dependency>
  <groupId>org.apache.hadoop</groupId>
  <artifactId>hadoop-hdfs</artifactId>
  <version>${hadoop.version}</version>
</dependency>
<dependency>
  <groupId>org.apache.hadoop</groupId>
  <artifactId>hadoop-common</artifactId>
  <version>${hadoop.version}</version>
</dependency>

<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-core</artifactId>
  <version>${flink.version}</version>
</dependency>
<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-streaming-java_2.11</artifactId>
  <version>${flink.version}</version>
</dependency>

<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-connector-jdbc_2.11</artifactId>
  <version>${flink.version}</version>
</dependency>

<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-java</artifactId>
  <version>${flink.version}</version>
</dependency>
```

```
<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-clients_2.11</artifactId>
  <version>${flink.version}</version>

</dependency>
<!-- https://mvnrepository.com/artifact/org.apache.flink/flink-table-api-java-bridge -->
<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-table-api-java-bridge_2.11</artifactId>
  <version>${flink.version}</version>
  <!--<scope>provided</scope>-->
</dependency>

<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-table-common</artifactId>
  <version>${flink.version}</version>
</dependency>

<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-table-planner_2.11</artifactId>
  <version>${flink.version}</version>
</dependency>

<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-table-planner-blink_2.11</artifactId>
  <version>${flink.version}</version>
</dependency>

<!-- https://mvnrepository.com/artifact/com.ververica/flink-connector-mysql-cdc -->
<dependency>
  <groupId>com.ververica</groupId>
  <artifactId>flink-connector-mysql-cdc</artifactId>
  <version>2.0.0</version>
</dependency>

<dependency>
  <groupId>mySQL</groupId>
  <artifactId>mySQL-connector-java</artifactId>
  <version>5.1.47</version>
```

```
</dependency>
<dependency>
  <groupId>log4j</groupId>
  <artifactId>apache-log4j-extras</artifactId>
  <version>1.2.17</version>
  <scope>compile</scope>
</dependency>

<dependency>
  <groupId>hang</groupId>
  <artifactId>hang</artifactId>
  <scope>system</scope>
  <systemPath>${project.basedir}/lib/hudi-flink-bundle_2.11-0.10.0-SNAPSHOT.jar</systemPath>
</dependency>

<dependency>
  <groupId>hangge</groupId>
  <artifactId>hangge</artifactId>
  <scope>system</scope>
  <systemPath>${project.basedir}/lib/hudi-cli-0.10.0-SNAPSHOT.jar</systemPath>
</dependency>

<dependency>
  <groupId>hangge</groupId>
  <artifactId>hangge</artifactId>
  <scope>system</scope>
  <systemPath>${project.basedir}/lib/hudi-hive-sync-bundle-0.10.0-SNAPSHOT.jar</systemPath>
</dependency>
<dependency>
  <groupId>hangge</groupId>
  <artifactId>hangge</artifactId>
  <scope>system</scope>
  <systemPath>${project.basedir}/lib/hudi-hadoop-mr-bundle-0.10.0-SNAPSHOT.jar</systemPath>
>
</dependency>
<!-- https://mvnrepository.com/artifact/org.apache.flink/flink-connector-kafka-0.10 -->
<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-connector-kafka_2.11</artifactId>
  <version>1.13.1</version>
</dependency>
<dependency>
```

```
<groupId>org.apache.kafka</groupId>
<artifactId>kafka-streams</artifactId>
<version>3.0.0</version>
</dependency>
<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-json</artifactId>
  <version>1.13.1</version>
</dependency>
<!-- https://mvnrepository.com/artifact/com.alibaba.ververica/flink-format-changelog-json -->
<dependency>
  <groupId>com.alibaba.ververica</groupId>
  <artifactId>flink-format-changelog-json</artifactId>
  <version>1.4.0</version>
</dependency>
</dependencies>
</project>
```

1.14.3. mysql-cdc-hudi

```
package com.doitedu.cdc.hudi.demo;
import org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;
import org.apache.flink.table.api.EnvironmentSettings;
import org.apache.flink.table.api.SqlDialect;
import org.apache.flink.table.api.TableResult;
import org.apache.flink.table.api.bridge.java.StreamTableEnvironment;

/**
 * flink-CDC 监控 mysql 表中数据的变化，将数据同步到 hudi 中
 */
public class C01_Mysql2Hudi {
    public static void main(String[] args) throws Exception {
        System.setProperty("HADOOP_USER_NAME", "root");
        EnvironmentSettings fsSettings = EnvironmentSettings.newInstance()
            .useBlinkPlanner()
            .inStreamingMode()
            .build();
        StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();
        env.setParallelism(1);
        StreamTableEnvironment tableEnv = StreamTableEnvironment.create(env, fsSettings);

        tableEnv.getConfig().setSqlDialect(SqlDialect.DEFAULT);
    }
}
```

```
// 数据源表
String sourceDDL =
    "CREATE TABLE tb_stu_mysql (\n" +
        "\tuuid VARCHAR(20),\n" +
        "\tid INT NOT NULL,\n" +
        "\tname VARCHAR(40),\n" +
        "\tdescription VARCHAR(40),\n" +
        "\tts TIMESTAMP(3)\n" +
        ") WITH (\n" +
        " 'connector' = 'jdbc',\n" +
        " 'url' = 'jdbc:mysql://doit01:3306/test', \n" +
        " 'driver' = 'com.mysql.cj.jdbc.Driver', \n" +
        " 'username' = 'root',\n" +
        " 'password' = 'root', \n" +
        " 'table-name' = 'tb_stu'\n" + // mysql test 数据库下的表
        ")";

// 输出目标表
String sinkDDL =
    "CREATE TABLE tb_stu_hudi(\n" +
        "\tuuid VARCHAR(20),\n" +
        "\tid INT NOT NULL,\n" +
        "\tname VARCHAR(40),\n" +
        "\tdescription VARCHAR(40),\n" +
        "\tts TIMESTAMP(3)\n" +
        ")\n" +
    // "PARTITIONED BY (partition")\n" +
    "WITH (\n" +
    "\t'connector' = 'hudi',\n" +
    "\t'path' = 'hdfs://doit01:8020/hudi_flink/tb_stu',\n" +
    "\t'table.type' = 'MERGE_ON_READ'\n" +
    ")";

// 简单的聚合处理
String transformSQL =
    "insert into tb_stu_hudi select * from tb_stu_mysql";
tableEnv.executeSql(sourceDDL);
tableEnv.executeSql(sinkDDL);
TableResult result = tableEnv.executeSql(transformSQL);
}
}
```

1. 14. 4. 测试加载 hudi 中的数据

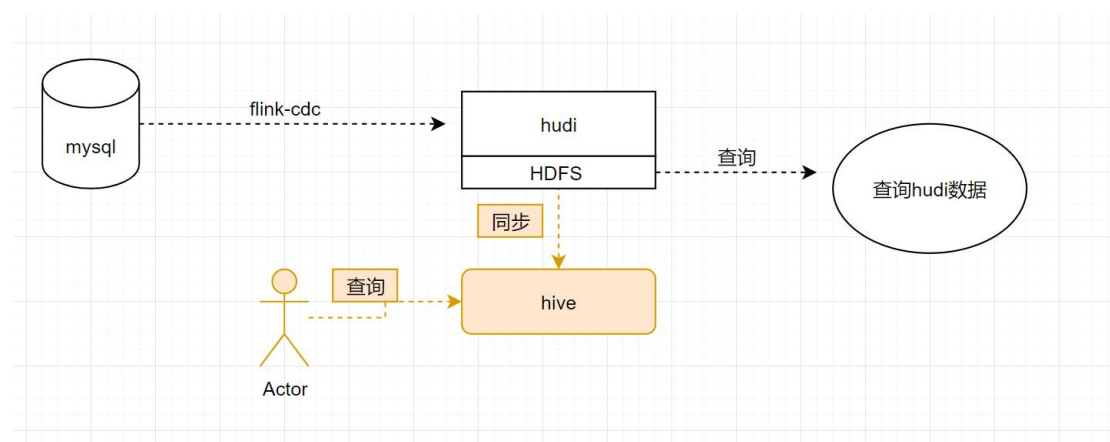

```
package com.doitedu.cdc.hudi.demo;
import org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;
import org.apache.flink.table.api.EnvironmentSettings;
import org.apache.flink.table.api.SqlDialect;
import org.apache.flink.table.api.TableResult;
import org.apache.flink.table.api.bridge.java.StreamTableEnvironment;
/**
 * @Date 2021/10/26
 * @Created by HANGGE
 * @Description
 * 读取 hudi 中同步 mysql 的数据
 */
public class C02_ReadHudi {

    public static void main(String[] args) throws Exception {
        EnvironmentSettings fsSettings = EnvironmentSettings.newInstance()
            .useBlinkPlanner()
            .inStreamingMode()
            .build();
        StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();
        env.setParallelism(1);
        StreamTableEnvironment tableEnv = StreamTableEnvironment.create(env, fsSettings);

        tableEnv.getConfig().setSqlDialect(SqlDialect.DEFAULT);
        String sourceDDL =
            "CREATE TABLE tb_stu_hudi(\n" +
            "    \tuuid VARCHAR(20),\n" +
            "    \tid INT NOT NULL,\n" +
            "    \tname VARCHAR(40),\n" +
            "    \tdescription VARCHAR(40),\n" +
            "    \tts TIMESTAMP(3)\n" +
            ")\n" +
            "WITH (\n" +
            "    \t'connector' = 'hudi',\n" +
            "    \t'path' = 'hdfs://doit01:8020/hudi_flink/tb_stu/',\n" +
            "    \t'table.type' = 'MERGE_ON_READ' )\n" ;
        tableEnv.executeSql(sourceDDL);
        TableResult result2 = tableEnv.executeSql("select * from tb_stu_hudi");
        result2.print();
    }
}
```

op	uuid	id	name	description	ts
+I	uuid002	1	ZSS	sanhaoxuesheng	2021-11-05 02:19:07.000
+I	uuid001	1	ZSS	sanhaoxuesheng	2021-11-05 02:19:04.000
+I	uuid003	2	lss	doitedu	2021-11-06 01:32:53.000

1.14.5. mysql-cdc-hudi-hive



环境准备：

将 hudi 的 jar 包添加到 hive 中

Hive Environment

- Import `hudi-hadoop-mr-bundle` into hive. Creating `auxlib/` folder under the root directory of hive, and moving `hudi-hadoop-mr-bundle-0.x.x-SNAPSHOT.jar` into `auxlib`. `hudi-hadoop-mr-bundle-0.x.x-SNAPSHOT.jar` is at `packaging/hudi-hadoop-mr-bundle/target`.
- When Flink sql client connects hive metastore remotely, `hive metastore` and `hiveserver2` services need to be enabled, and the port number need to be set correctly. Command to turn on the services:

hadoop-2.7.3.tar.gz	2021/10/30 14:09	360压缩	209,075 KB
apache-hive-2.3.1-bin.tar.gz	2021/10/30 14:03	360压缩	213,776 KB
scala-2.12.10.tgz	2021/10/30 13:43	360压缩	20,185 KB
hudi-0.9.0.src.tgz	2021/10/30 13:37	360压缩	2,040 KB
apache-maven-3.8.3-bin.tar.gz	2021/10/29 21:12	360压缩	8,831 KB
hudi-flink-bundle-2.11-0.10.0-SNAPSHOT.jar	2021/10/28 21:07	JAR 文件	71,882 KB
hudi-hive-sync-bundle-0.10.0-SNAPSHOT.jar	2021/10/28 17:01	JAR 文件	3,375 KB
hudi-hadoop-mr-bundle-0.10.0-SNAPSHOT.jar	2021/10/28 17:01	JAR 文件	17,023 KB
hudi-master (1).zip	2021/10/28 16:44	360压缩 ZIP 文件	9,306 KB
谷歌浏览器.exe	2021/10/28 9:39	应用程序	1,725 KB

```

[root@doit01 lib]# ll | grep hudi-
-rw-r--r--. 1 root root 73606625 oct 28 21:07 hudi-flink-bundle-2.11-0.10.0-SNAPSHOT.jar
-rw-r--r--. 1 root root 17430622 oct 28 17:01 hudi-hadoop-mr-bundle-0.10.0-SNAPSHOT.jar
-rw-r--r--. 1 root root 3455733 oct 28 17:01 hudi-hive-sync-bundle-0.10.0-SNAPSHOT.jar
  
```

启动元数据服务 `hive --service metastore &` {后台启动}

重启远程连接服务 `hiveserver2 &` {后台启动}

```

public static void main(String[] args) {
    // 获取flinksql的执行环境
    System.setProperty("HADOOP_USER_NAME", "root");
  }
  
```

```
EnvironmentSettings fsSettings = EnvironmentSettings.newInstance()
    .useBlinkPlanner()
    .inStreamingMode()
    .build();

StreamExecutionEnvironment env =
StreamExecutionEnvironment.getExecutionEnvironment();
env.setParallelism(1);
StreamTableEnvironment tableEnv = StreamTableEnvironment.create(env, fsSettings);
tableEnv.getConfig().setSqlDialect(SqlDialect.DEFAULT);

// 1 监控 mysql 数据的变化
String fromMySQL = "create table tb_stu_mysql(\n" +
    "uuid varchar(20),\n" +
    "id int,\n" +
    "name varchar(40),\n" +
    "description varchar(40),\n" +
    "ts` timestamp(3)\n" +
    ")with(\n" +
    "'connector' = 'jdbc',\n" +
    "'url' = 'jdbc:mysql://doit01:3306/flink_cdc',\n" +
    "'driver' = 'com.mysql.cj.jdbc.Driver',\n" +
    "'username' = 'root',\n" +
    "'password' = 'root',\n" +
    "'table-name' = 'tb_stu'\n" +
    ")";

tableEnv.executeSql(fromMySQL);

// 2 将数据写入到 hudi 同步到 hive
String sink2HudisyncHive = "create table tb_stu(\n" +
    "uuid varchar(20),\n" +
    "id int,\n" +
    "name varchar(40),\n" +
    "description varchar(40),\n" +
    "ts` timestamp(3)\n" +
    ")\n" +
    // "partitioned by (name) \n" +
    "with (\n" +
    "'connector' = 'hudi',\n" +
    "'path' = 'hdfs://doit01:8020/flink_hudi/tb_stu',\n" +
    "'table.type' = 'COPY_ON_WRITE',\n" +
    // 将数据同步到 hive
    "'write.precombine.field' = 'ts',\n" +
    "'write.recordkey.field' = 'id',\n" +
```

```

        ""hive_sync.enable' = 'true',\n" +
        ""hive_sync.mode' = 'jdbc',\n" +
        // ""hive_sync.partition_fields'='name', \n"+
        //
        ""hive_sync.partition_extractor_class'='org.apache.hudi.hive.MultiPartKeysValueExtractor',\n"
+
        ""hive_sync.metastore.uris' = 'thrift://doit01:9083',\n" +
        ""hive_sync.jdbc_url' = 'jdbc:hive2://doit01:10000',\n" +
        ""hive_sync.db' = 'my_db',\n" +
        ""hive_sync.table'='tb_stu',\n" +
        ""hoodie.datasource.hive_sync.partition_fields'='name',\n" +
        ""hive_sync.username' = 'root' \n" +
        ")";

tableEnv.executeSql(sink2HudisyncHive);
String doSink = "insert into tb_stu  select * from tb_stu_mysql";
tableEnv.executeSql(doSink).print();

//3 执行同步数据
// tableEnv.executeSql(doSink2Hudi);

```

当 mysql 中的数据发生变化的时候，我们可以同步变化的业务数据到 hive！

实现了 hive 中数据的逐行删除，更新，插入

tb_stu_hoodie_commit_time	tb_stu_hoodie_commit_seqno	tb_stu_hoodie_record_key	tb_stu_hoodie_partition_path	tb_stu_hoodie_file_name	tb_stu_uuid	tb_stu_id	tb_stu_name	tb_stu
20211111223712	20211111223712_3_1	uid4002		03c0e45c-61d9-4824-9cc2-883f548ea4f4_3-4-0_20211111223712.parquet	uid4002	2	LSS	
1636628287000	20211111223712_3_2	uid4001		03c0e45c-61d9-4824-9cc2-883f548ea4f4_3-4-0_20211111223712.parquet	uid4001	1	ZSS	
20211111223712	20211111223712_3_3	uid4004		03c0e45c-61d9-4824-9cc2-883f548ea4f4_3-4-0_20211111223712.parquet	uid4004	4		
1636628287000	20211111223712_3_4	uid4003		03c0e45c-61d9-4824-9cc2-883f548ea4f4_3-4-0_20211111223712.parquet	uid4003	3	MM	
1636628287000	20211111223712_3_5	uid4005		03c0e45c-61d9-4824-9cc2-883f548ea4f4_3-4-0_20211111223712.parquet	uid4005	5	LYD	

col_name	data_type	comment
# col_name	data_type	comment
_hoodie_commit_time	string	
_hoodie_commit_seqno	string	
_hoodie_record_key	string	
_hoodie_partition_path	string	
_hoodie_file_name	string	
uuid	string	
id	int	
name	string	
description	string	
ts	bigint	
# Detailed Table Information		
Database:	my_db	
OwnerType:	USER	
Owner:	root	
CreateTime:	Thu Nov 11 22:33:22 CST 2021	
LastAccessTime:	UNKNOWN	
Retention:	0	
Location:	hdfs://doit01:8020/flink_hudi/tb_stu	
Table Type:	EXTERNAL_TABLE	
Table Parameters:		
	EXTERNAL	
	bucketing_version	2
	last_commit_time_sync	20211111223712
	numFiles	2
	spark.sql.sources.provider	hudi
	spark.sql.sources.schema.numParts	1
	spark.sql.sources.schema.part.0	{\n"type":\n"struct",\n"fields":\n[\n{\n"name":\n"_hoodie_com
	_commit_seqno"\n"type":\n"string",\n"nullable":true,\n"metadata":\n{\n{\n"name":\n"_hoodie_record_key"\n"type":\n"string",\n"nullable":true,\n"metadata":\n{\n{\n"name":\n"_hoodie_file_name"\n"type":\n"string",\n"nullable":true,\n"metadata":\n{\n{\n"name":\n"_description"\n"type":\n"str	
	totalSize	870687
	transient_lastDdlTime	1636641202
	NULL	NULL
	NULL	NULL
# Storage Information		
SerDe Library:	org.apache.hadoop.hive.q1.io.parquet.serde.ParquetHiveSerDe	NULL
InputFormat:	org.apache.hudi.hadoop.HoodieParquetInputFormat	NULL
OutputFormat:	org.apache.hadoop.hive.q1.io.parquet.MapredParquetOutputFormat	NULL
Compressed:	No	NULL
Num Buckets:	-1	NULL
Bucket Columns:	[]	NULL
Sort Columns:		NULL
Storage Desc Params:		NULL
	hoodie.query.as.ro.table	false
	path	hdfs://doit01:8020/flink_hudi/tb_stu
	serialization.format	1

44 rows selected (0.107 seconds)


```
        "id int ,\n" +
        "name varchar(40) ,\n" +
        "description varchar(40),\n" +
        "`ts` timestamp(3)\n" +
        ")with(\n" +
        "'connector' = 'jdbc' ,\n" +
        "'url' = 'jdbc:mysql://doit01:3306/flink_cdc' ,\n" +
        "'driver' = 'com.mysql.cj.jdbc.Driver' ,\n" +
        "'username' = 'root' ,\n" +
        "'password' = 'root' ,\n" +
        "'table-name' = 'tb_stu'\n" +
        ")";

tableEnv.executeSql(fromMySQL);

//2 将数据写入到 hudi
String sink2kafka = "create table tb_kafka(\n" +
        "uuid varchar(20) ,\n" +
        "id int ,\n" +
        "name varchar(40) ,\n" +
        "description varchar(40),\n" +
        "`ts` timestamp(3)\n" +
        ")\n" +
        "with (\n" +
        "'connector' = 'kafka' ,\n" +
        "'topic' = 'ods' ,\n" +
        "'properties.bootstrap.servers' = 'doit01:9092,doit02:9092,doit03:9092' ,\n" +
        "'format' = 'changelog-json' \n" +
        ")";

tableEnv.executeSql(sink2kafka);

String doSink2Kafka = "insert into tb_kafka select * from tb_stu_mysql";

//3 执行同步数据
tableEnv.executeSql(doSink2Kafka);

}
}
```

```
root@doit01:~/hudi# ./kafka-console-consumer.sh --bootstrap-server doit01:9092 --from-beginning --topic ods
{"data":{"uuid":"uuid001","id":1,"name":"ZSS","description":"good coder","ts":"2021-11-11 10:57:48"},"op":"+I"}
{"data":{"uuid":"uuid002","id":2,"name":"LSS","description":"bad coder","ts":"2021-11-11 10:58:07"},"op":"+I"}
{"data":{"uuid":"uuid003","id":3,"name":"WW","description":"bad coder","ts":"2021-11-11 11:07:17"},"op":"+I"}
{"data":{"uuid":"uid004","id":4,"name":"XIAOLIU","description":"good teacher","ts":"2021-11-12 12:36:48.355"},"op":"+I"}
{"data":{"uuid":"uid005","id":5,"name":"LYD","description":"good teacher","ts":"2021-11-12 12:35:50"},"op":"+I"}
```

1. 14. 7. kafka 数据落地 hudi

- **group-offsets**: start from committed offsets in ZK / Kafka brokers of a specific consumer group.
- **earliest-offset**: start from the earliest offset possible.
- **latest-offset**: start from the latest offset.
- **timestamp**: start from user-supplied timestamp for each partition.
- **specific-offsets**: start from user-supplied specific offsets for each partition.

1. 15. 使用姿势

1. 15. 1. 近实时读取

Hudi 在各种读取数据方面也有很多好处, Hudi 在 DFS 分布式存储系统上存储数据强制执行了最小文件大小, 这样有助于解决 HDFS 和存储上的小文件问题, 显著的提升了查询性能。并且 Hudi 每次提交数据的方式都是原子性的, 这样也避免了查询时受到部分写入的影响。将外部各种 OLTP 的数据源(比如日志数据、数据库、外部源)写入到 Hudi 中也是一个常见的问题, Hudi 存储这些数据, 原始数据层的构建也是非常关键。

对应 RDBMS 这种关系型数据库的数据写入, Hudi 提供了 Upserts 操作来提供增量修改或新增, 而不是采用昂贵的且低效的批量加载。使用 Debezium 或 Kafka Connect 或 Sqoop 等工具将数据导入到 Hudi 对应 DFS 上的表是非常常见的一种方案。对于像 NoSql 这样的数据库(Cassandra / Voldemort / HBase)即使中等规模的数据量会存储十亿行左右, 所以使用完全批量加载的方式根本不行, 如果读取要跟上数据的高更新的变化量, 则需要更有效的方法。即使对于像 Kafka 这样的不可变数据源, 通常也需要根据 DFS 上存储的内容对传入事件进行重复数据的删除。Hudi 通过使用不同类型的索引来快速解决这个问题。

所有的一切都可以通过 Hudi DeltaStreamer 工具无缝实现, 该工具 Hudi 官方一直在尝试增加更多数据源, 该工具提供一个连续模式, 可以异步地自我管理集群/压缩, 而不会阻止数据的写入, 显著提高数据新鲜度。

1. 15. 2. 删除和更新数据

Hudi 还提供了删除存储在数据中的数据的能力, 更重要的是提供了处理大量写入放大(wa)的有效方法, 这些通过 Merge On Read 表类型基于 user_id(任何辅助键)随件删除产生的结果。Hudi 可以基于日志提供优雅的并发控制, 保证数据的写入和读取可以持续发生, 因为后台压缩作业分摊数据的重写和强制删除所需要的成本。

Hudi 还解锁了数据聚类等特殊功能, 允许用户优化数据布局来进行删除。具体来说, 用户可以基于 user_id(辅助键)对旧的事件日志数据进行聚类, 这样评估数据需要删除的数据就可以快速的定位, 对于分区则在时间戳上进行了聚类优化, 提高查询性能。

1. 15. 3. 增量处理

在传统数据仓库中, 整个数仓的工作流可能会发生数据延迟的问题, 比如上游工作流 U 每

小时会在 Hive 建立分区, 每小时结束时使用该处理时间作为 event time, 提供 1 小时的数据有效新鲜度, 然后下游工作流 D 在 U 完成后立即启动, 并在接下来一小时内进行自己的处理, 将有效延迟增加到了 2 小时。这样的例子忽略了数据迟到现象, 即 processing_time 和 event time 并不是同一时间而是分开的, 比如移动设备或者传感器间歇性的连接所造成, 这种现象并不是异常而是常态, 那么在这种情况下, 保证正确性的唯一补救措施是每小时一遍一遍得重新处理那些迟到的数据, 这样可能会严重危害到整个生态系统的效率。

那么 Hudi 也提供了解决方案, 它提供一种以记录粒度 (不是目录/分区) 从上有 Hudi 表中消费数据 (包括后期迟到数据) 的方法, 可以有效地更新/协调后期数据到达, 上下游两张表的调度可以更频繁, 例如 15 分钟, 并可以在下游表提供 30 分钟的端延迟。为了实现这一目标, Hudi 采纳了 spark streaming、Kafka、Flink、Oracle Streaming 等技术框架的概念。

1. 15. 4. 分析存储

数据的存储和分析一般我们分为两类数据, 实时处理和离线批量处理。通常实时处理由 Druid、Mysql 或 clickhouse 提供支持, 并且有 kafka 或 pulsar 提供服务, 这种模型非常昂贵。

如果数据会在很晚之后才被写入到数据湖中, 那么就需要进行批处理, Hudi 也提供对 Persto/Spark Sql 等交互时 Sql 引擎的访问, 这些引擎可以轻松横向扩展, 并在几秒钟返回查询结果。

与实时数据集市相比, 通过将数据新鲜度缩短到几分钟, Hudi 可以为大量数据应用程序提供更高效的替代方案。